BASE TEÓRICA Y CONTRATO DE INTEGRACIÓN

CU01 · Importación | CU02 · Índice de sobrecarga | CU03 · Informes y alertas

Proyecto del Grupo 8 · Next.js + TypeScript · Contrato de implementación del primer prototipo. Las decisiones operativas de este documento se aplican al P0; los valores demostrativos no constituyen validación del modelo ni aprobación docente.

Cómo usar el documento

Para ubicar tu trabajo: Tareas Resumen → tu tarjeta → pestaña CU correspondiente → Contratos y pruebas. Para entender el sistema: Objetivo y Modelo → CU01 → CU02 → CU03 → Contratos y pruebas. Tareas en Detalle desarrolla las mismas tareas y conserva el plan orientativo; Sugerencias explica decisiones y evolución.

Reglas de lectura

La matriz de alcance de esta pestaña define qué se entrega en cada etapa. CU02 define las matemáticas; Contratos y pruebas define tipos, API, estados y ejemplos; las tareas aplican esas reglas sin redefinirlas. Los IDs T01–T16 identifican tareas: no indican orden cronológico. T15 se divide en T15a y T15b (17 tareas en total).

Los parámetros v1 quedan fijados como referencia de desarrollo y pruebas. Su validez operativa debe contrastarse con la profesora y con datos adecuados antes de un uso real. Este documento no modifica por sí solo la Entrega 3 formal ni las tarjetas de Trello.

1. OBJETIVO Y DECISIONES DE ARQUITECTURA

El sistema identifica INDICADORES de posible sobrecarga laboral a partir de datos operativos. No diagnostica burnout, no trata información de salud ni toma decisiones sobre personal. El responsable autorizado evalúa las recomendaciones y decide fuera del sistema.

- Flujo troncal: RRHH selecciona planilla → CU01 valida e importa → CU02 calcula por empleado y período → CU03 registra una alerta si corresponde y produce información consultable/exportable.
- Tecnología acordada: Next.js con TypeScript. La arquitectura de tres capas se mantiene conceptualmente: interfaz, servicios/reglas de negocio y persistencia; ya NO se implementará el backend Python de la Entrega 3.
- CU01, CU02 y CU03 son módulos separados y conectados mediante contratos tipados. CU02 y CU03 se ejecutan en el servidor como parte del procesamiento de una importación; no requieren pulsar un botón por empleado.

- Alcance P0: demo local con datos ficticios, CSV semanal, MySQL/Prisma, reglas v1, evaluación e historial, informe, alertas internas y tablero simple. XLSX y exportaciones quedan para el sistema final; simulador de correo, roles demo y auditoría ampliada son P1. La matriz de alcance al final de esta pestaña es la referencia única.
- Persistencia P0: MySQL mediante Prisma. Los arrays transportan datos entre servicios; la base conserva registros, versiones, evaluaciones, trabajos de procesamiento y alertas. CU02 matemático es una función pura y puede probarse con objetos simulados.

2. MODELO DE DATOS COMPARTIDO (CONTRATOS)

2.1 Identidad, tiempo y trazabilidad

- Empleado: employeeId (string, código estable y único), teamId (string), responsableId (string o null), activo (boolean). El nombre es opcional para el archivo; se obtiene del catálogo de empleados para evitar inconsistencias.
- Período: periodStart y periodEnd son fechas ISO YYYY-MM-DD sin hora. Cada fila cubre lunes a domingo, ambos inclusive, y solo se admite si ese domingo ya terminó en America/Argentina/Buenos_Aires (zona fija del prototipo). Los timestamps se guardan en UTC. No convertir fechas de período mediante la zona del navegador.
- Importación: importId, sourceName, sourceHash, schemaVersion='indicadores-v1', uploadedAt y uploadedBy='demo-local' en P0. status = VALIDADA | RECHAZADA | CANCELADA | CONFIRMADA. Validar guarda únicamente metadatos y vista previa temporales; no crea RegistroLaboral ni Evaluacion. ProcessingJob separado: PENDIENTE | PROCESANDO | COMPLETADO | FALLIDO; detalle en Contratos y pruebas.
- Registro del período: clave lógica employeeId + periodStart + periodEnd, con una sola versión vigente. Conservar sourceImportId, dataVersion, fingerprint de contenido canónico y, para futuras correcciones, supersedesRecordId. En P0 un conflicto de contenido se rechaza: no existe sustitución desde la importación.

2.2 Indicadores por empleado y período

- horasTrabajadas?: number (horas reales del período, ≥ 0; dato auxiliar de control y visualización).
- horasExtra: number (horas extra del período, ≥ 0; variable 1 del índice). No derivar siempre horas extra de horasTrabajadas, porque dependen de jornada pactada y reglas aplicables.
- diasConsecutivosSinDescanso: number (entero ≥ 0; cantidad de jornadas consecutivas al CIERRE del período, incluso si la racha comenzó antes; variable 2). NO sumar días sin descanso de cada semana.

- tareasActivas: entero ≥ 0. Nombre heredado que significa TOTAL DE TAREAS DEL PERÍODO: unión, sin repetir task_id, de las tareas abiertas al inicio y las asignadas durante la semana, incluyendo las que se completaron antes del cierre. No significa solo tareas que siguen abiertas. En P0 no se admiten reasignaciones/cancelaciones intermedias: la fuente debe consolidar previamente un responsable único para cada tarea del período.
- tareasPendientes: entero ≥ 0, subconjunto del total anterior sin completar al cierre. tareasVencidas: subconjunto de pendientes con vencimiento anterior al día siguiente al cierre. Debe cumplirse 0 ≤ vencidas ≤ pendientes ≤ activas. Completadas = activas − pendientes. Con 10 totales, 7 pendientes y 5 vencidas hay 3 completadas, 2 pendientes no vencidas y 5 vencidas.
- vacacionesPendientesDias: number (entero ≥ 0; saldo de vacaciones disponible/no utilizado al cierre, según RRHH; variable 4). Su sola existencia no significa riesgo alto.
- diasDesdeUltimasVacaciones?: number | null (entero ≥ 0 o null si no está disponible; contexto interpretativo, no se pondera en la fórmula inicial).
- Campos admitidos en la planilla P0: los nueve obligatorios y los dos opcionales de Sugerencias §6.2. horasPactadas y observaciones no forman parte del contrato v1; si se incorporan después, requerirán nueva versión de esquema.

2.3 Objetos que intercambian los casos de uso

Contratos tipados obligatorios para integrar. Esta sección resume su función; las definiciones exactas y las respuestas HTTP están en «Contratos y pruebas». Gonzalo publica los tipos compartidos en T01 y Eros revisa la correspondencia con CU02.

`RegistroLaboral: identidad + semana + seis métricas obligatorias (HE, DS, total de tareas, pendientes, vencidas y VP) + dos auxiliares opcionales. El servidor agrega recordId, sourceImportId, dataVersion y fingerprint. Toda magnitud es number finito; horas permiten decimales y días/tareas son enteros.`

`ResultadoImportacion: importId, status, insertados, omitidos y pares afectados {employeeId,periodStart,periodEnd}; procesamiento con jobId y estado independiente. Los errores de validación se devuelven con fila, campo, código y detalle. No informar registros «aceptados» como si estuvieran confirmados en la vista previa.`

`Evaluacion: resultado terminado e inmutable, con entradas y reglas en snapshot, scores, contribuciones, ranking,`

`indice:number, nivel:BAJO|MEDIO|ALTO|INCOMPLETO y persistencia explicada. INCOMPLETO en P0 significa falta de semana anterior; conserva índice numérico. Una falla técnica pertenece a ProcessingJob y no inventa una evaluación ni un nivel. Ver CU02 §4.4.`

`Alerta: una por evaluationId, destinatarioId nullable, causas, sugerencias y vigencia ACTIVA|SUPERADA. Canal INTERNO en P0, con estadoEnvio=NO_APLICA. En P1 el canal SIMULADO registra PENDIENTE|ENVIADO|FALLIDO|SIN_DESTINATARIO e intentos. ENVIADO en ese canal siempre significa simulación.`

Trazabilidad mínima P0: autor demo, archivo/hash, hora, esquema, reglas, versión de datos, antecedentes usados y resultado. T15b amplía consultas y eventos de auditoría; no posterga estos metadatos básicos. Las evaluaciones históricas se conservan; las consultas normales muestran solo la vigente por empleado y semana.

MATRIZ ÚNICA DE ALCANCE

P0 es el corte de entrega de las 14 tareas imprescindibles. P1 tiene 3 tareas complementarias. Sistema final indica ampliación pendiente sin compromiso para la demo. El P0 no cumple por sí solo todos los RF de la Entrega 3.

Función	**P0**	**P1**	**Sistema final**
CSV semanal + validación atómica	Obligatorio	Mantener	Mantener
MySQL/Prisma, historial y jobs	Obligatorio	Mantener	Mantener
Índice, ranking, informe y tablero simple	Obligatorio	Mantener	Ampliar
Alertas internas y destinatarios del seed	Obligatorio	Mantener	Ampliar
Metadatos, snapshots y reglas por script	Obligatorio	Mantener	Mantener
Roles/usuarios de demostración	Fuera	T15a	Autenticación productiva
Simular correo y reintentos	Fuera	T12	Correo real

Eventos/consulta de auditoría ampliada	Fuera	T15b	Ampliar
Importar XLSX; exportar CSV/XLSX/PDF	Fuera	Fuera	Pendiente
Editor web de pesos y umbrales	Fuera	Fuera	Pendiente
Reemplazar una carga confirmada	Rechazar conflicto	Rechazar conflicto	Corrección autorizada/versionada
CRUD y vista del empleado	Seed solamente	Sin CRUD completo	Pendiente
Jornadas/tareas/licencias detalladas (RF03)	Resumen solamente	Resumen solamente	Pendiente de definición formal

3. CU01 · CARGAR E IMPORTAR DATOS LABORALES

3.1 Entradas y salidas

- Actor P0: operador demo local equivalente a RRHH, con datos ficticios. Entrada externa obligatoria: CSV indicadores-v1. XLSX y carga manual corresponden al sistema final; roles por sesión se prueban en P1. La matriz de Objetivo y Modelo fija el alcance.
- Entrada técnica: File en la interfaz; archivo o bytes recibidos por ruta de servidor; filas crudas temporales (Record<string, string | number | null>[]); catálogo de empleados registrado en base de datos.
- Salida hacia CU02: RegistroLaboral[] normalizado, organizado por employeeId y período SOLO después de confirmación y guardado correcto. Salida hacia RRHH: ResultadoImportacion con resumen y errores identificados.
- Salida persistente: metadatos de importación, registros confirmados y trabajo de procesamiento. Validar solo conserva una previsualización temporal durante 30 minutos; cancelar no crea datos laborales. Nunca transmitir a CU02 filas inválidas ni depender únicamente de arrays en memoria.

3.2 Procedimiento obligatorio

1. El operador selecciona CSV UTF-8 separado por comas, máximo 5 MiB y 1.000 filas de datos. Se admite BOM, LF o CRLF y campos entre comillas según CSV; números con punto decimal. Rechazar archivo vacío, bytes nulos, formato no CSV y encabezados repetidos/desconocidos. XLSX no se ofrece en el selector P0.
2. Parsear todas las filas; recortar espacios de encabezados y celdas sin cambiar mayúsculas de IDs; mapear snake_case a camelCase según Contratos y pruebas. Ignorar filas completamente vacías. No interpretar fórmulas, separadores de miles ni fechas locales. No redondear métricas.
3. Se verifica existencia y ámbito del employeeId; luego formato, unicidad, período y consistencia de métricas. Los errores citan fila, campo, código y explicación.
4. Comparar clave empleado+semana con lote y base: duplicado interno → error; histórico idéntico → OMITIDO; histórico distinto → CONFLICTO y rechazo de todo el lote. P0 no permite sobrescribir. La huella se calcula sobre datos canónicos, sin hash de archivo, importId ni timestamps.
5. Devolver vista previa con importId, vencimiento, filas nuevas/omitidas, errores y warnings. Si hay un error, status=RECHAZADA y Confirmar queda deshabilitado. Cancelar descarta datos temporales; una carga válida no confirmada vence a los 30 minutos.

6. Confirmar por importId: revalidar catálogo y duplicados contra el estado actual de la base, y en una transacción insertar solo las filas nuevas, marcar CONFIRMADA y crear ProcessingJob PENDIENTE si hay afectados. Sin parciales. Una segunda confirmación devuelve el mismo resultado, sin insertar ni disparar de nuevo.
7. Tras confirmar, procesar en servidor los pares nuevos y su semana siguiente ya existente, por empleado y orden ascendente de fecha. CU02 guarda evaluación y CU03 produce informe/alerta para TODO nivel; la alerta se crea solo para ALTO. Si falla después del commit, mantener la carga confirmada y job FALLIDO reintentable. El tablero distingue proceso pendiente de resultado INCOMPLETO.

3.3 Reglas de validación y duplicados (RF03 / RF04)

- Obligatorios: employeeId, inicio/fin de período, horasExtra, diasConsecutivosSinDescanso, tareasActivas, tareasPendientes, tareasVencidas, vacacionesPendientesDias. Los opcionales nunca sustituyen silenciosamente a los obligatorios.
- Fechas: YYYY-MM-DD real, inicio lunes, fin domingo exactamente 6 días después, semana cerrada en la zona definida y sin solapamientos por empleado. Las dos fechas son inclusivas; no aceptar semanas desplazadas de martes a lunes.
- Cantidades: no negativas, finitas (sin NaN/Infinity), días y cantidades de tareas enteros; horas pueden ser decimales. Validar tareasVencidas ≤ tareasPendientes ≤ tareasActivas. Coherencia DS: si existen semanas adyacentes en lote/base y DS de la posterior ≥7, debe ser DS_anterior+7; si es menor que 7, hubo descanso y no se exige esa igualdad. Comprobar también el sucesor ya guardado al importar antecedentes. Una contradicción es RANGO_INVALIDO y rechaza el lote; sin antecedente no se inventa la racha.
- Identidad: empleado existente, activo/en cobertura, asociado a un equipo; un registro por empleado+período en una misma importación. IDs repetidos en períodos distintos sí son válidos.
- Duplicado dentro del archivo: misma clave empleado+inicio+fin → error. Duplicado histórico idéntico: informar y omitir sin nueva evaluación. Misma clave con valores distintos: conflicto, no reemplazar sin autorización específica.
- Horas: 0 ≤ horasExtra ≤ 168 y, si horasTrabajadas existe, horasExtra ≤ horasTrabajadas ≤ 168. Más de 80 h trabajadas o 40 h extra produce warning sin rechazo si cumple los límites físicos. DS puede superar 7 porque es una racha acumulada; no truncarlo al tamaño de la semana.
- Vacaciones y descansos: días pendientes deben provenir de una fuente administrativa; no deducir saldos legales a partir de antigüedad, y no confundir vacaciones con descansos semanales.

- Si una fila falla, rechazar la confirmación del lote completo para cumplir la atomicidad del RF04/RNF04; mostrar todos los errores detectados de una sola vez.

3.4 Excepciones y criterios de aceptación

1. A1, paso 1: formato distinto de CSV, archivo vacío/excesivo o encabezados inválidos → error con código y mensaje. El rechazo por formato no crea registros laborales. XLSX se probará al incorporar esa ampliación.
2. A2, pasos 3–4: empleado inexistente, fecha inválida, dato inconsistente o duplicado conflictivo → lista de errores y ninguna modificación.
3. A3, paso 5: cancelar → no se guardan registros laborales; los metadatos pasan a CANCELADA. Un importId vencido requiere validar de nuevo.
4. A4, pasos 6–7: fallo antes del commit → rollback; fallo después → importación CONFIRMADA y job FALLIDO. Reintentar job no vuelve a importar ni duplica evaluación/alerta.

Aceptación P0 CU01: CSV con 50 empleados y semanas cerradas; una fila inválida impide guardar todo el lote; archivo repetido omite filas idénticas; doble confirmación es segura; un conflicto no sustituye datos; CU02 procesa exactamente los pares afectados y sus sucesores inmediatos existentes.

4. CU02 · CALCULAR ÍNDICE DE RIESGO DE SOBRECARGA

4.1 Entradas, salidas y secuencia

- Entrada: un RegistroLaboral confirmado, registro de la semana inmediatamente anterior si existe y ConfiguracionIndice aplicable al inicio de la semana actual. El motor calcula sin Prisma ni Next.js. El servicio de T08 obtiene datos, guarda evaluación y llama CU03.
- Salida: Evaluacion con índice numérico, cuatro scores, contribuciones, ranking completo, nivel y explicación de persistencia. El informe se genera para todos los niveles. Un error de contrato/reglas devuelve error técnico; no se confunde con INCOMPLETO.

1. Validar registro y reglas. Elegir la configuración de mayor effectiveFrom ≤ periodStart. Para P0 se usa v1 desde 2026-01-05. Buscar la semana cuyo periodStart sea exactamente 7 días anterior, no la última fila disponible.
2. Calcular las cuatro variables numéricas de acuerdo con las definiciones y umbrales.
3. Normalizar cada variable a escala 0–100 y aplicar los pesos.
4. Determinar los factores excedidos y comprobar persistencia en períodos consecutivos.
5. Clasificar y guardar snapshot completo de datos actuales, datos anteriores usados o ausencia explícita, reglas, resultado y processingFingerprint. No almacenar un índice null por falta de historial.
6. Persistir la evaluación y solicitar CU03 para construir el informe, incluso si el nivel es INCOMPLETO. Solo ALTO crea alerta. Errores de persistencia/cálculo mantienen job FALLIDO para reintento; no se publican como BAJO.

4.2 Definición concreta de las cuatro variables

- HE (horas extra): cantidad de horas extra de la SEMANA. Valor ≥ 0. Umbral inicial ilustrativo U_HE = 8 h/semana.
- DS (días seguidos): días trabajados consecutivos al final de esa semana, incluyendo antecedentes. Umbral inicial U_DS = 6 días.

CT · Proporción de pendientes y vencidas

A = tareasActivas: total de tareas que estuvieron bajo responsabilidad en la semana, incluidas las completadas; P = tareasPendientes al cierre; V = tareasVencidas al cierre. V ⊆ P ⊆ A.

```
CT = 0,60 × (V / A) + 0,40 × (P / A)
```

Si A=0, CT=0 solo cuando P=V=0. Si no se cumple 0≤V≤P≤A, CU01 rechaza el registro y el motor devuelve error de contrato. El peso adicional de V es intencional: una vencida aporta a ambos términos.

Umbral v1: U_CT=0,35. CT mide acumulación relativa, no esfuerzo ni productividad. A=10, P=10 y V=0 da CT=0,40 y supera este umbral: es el comportamiento deliberado de la regla demostrativa, no prueba de sobrecarga. Por sí solo no basta para ALTO.

VP (vacaciones pendientes): saldo de días al cierre. Umbral inicial U_VP = 10 días. Este indicador es una señal contextual de recuperación pendiente, no evidencia concluyente de sobrecarga.

4.3 Normalización, fórmula y pesos iniciales

Factor	Valor umbral	Peso	Lectura
HE	8 h / semana	30 %	Horas extra
DS	6 días	30 %	Racha al cierre
CT	0,35 (proporción)	25 %	Pendientes y vencidas
VP	10 días	15 %	Saldo vacaciones

Paso A · Normalizar cada variable

```
score_i = min(100; 50 × Xi / Ui)
```

Xi es el valor observado y Ui su umbral positivo. En TypeScript min equivale a Math.min. Alcanzar Ui produce score=50; duplicarlo produce score=100. «Excedida» significa Xi > Ui, estrictamente; Xi=Ui no cuenta como excedida.

Paso B · Ponderar sin redondear

```
contribución_i = peso_i × score_i
```

Los pesos HE=0,30, DS=0,30, CT=0,25 y VP=0,15 suman 1. Cada contribución se expresa en puntos del índice; no en porcentaje de causalidad.

Paso C · Obtener el índice

```
I = 0,30 × S_HE + 0,30 × S_DS + 0,25 × S_CT + 0,15 × S_VP
```

```
indice = Math.round((I + Number.EPSILON) × 100) / 100
```

S_HE, S_DS, S_CT y S_VP son los scores normalizados de cada factor. Ese indice con dos decimales es el valor contractual usado para comparar con 40 y 70. CT,

scores, contribuciones y comparación Xi>Ui usan precisión completa. El ranking se ordena por contribución sin redondear; desempate fijo HE → DS → CT → VP. Redondear solo su presentación a dos decimales; la suma de valores mostrados puede diferir 0,01 del índice.

Ejemplo · HE=16 h, DS=9 días, A=10, P=7, V=5 y VP=14 días

$$CT = 0{,}60 \times 5/10 + 0{,}40 \times 7/10 = 0{,}58$$

Scores: HE=100; DS=75; CT=82,857142…; VP=70.

Contribuciones: HE=30; DS=22,5; CT=20,714285…; VP=10,5.

Índice = 83,71. Ranking HE → DS → CT → VP. Sin semana anterior el nivel es INCOMPLETO. En el ejemplo integral de Contratos y pruebas, la semana siguiente tiene DS=16 e índice 91,21; ambas cumplen persistencia y la segunda queda ALTO.

4.4 Clasificación exacta y persistencia

Aplicar este orden, sin excepciones implícitas:

0. Si el registro o las reglas son inválidos, devolver error; no guardar Evaluacion. CU01 ya impide faltantes obligatorios en P0.

1. Calcular siempre el índice actual si los datos/reglas son válidos. Si no existe registro válido de la semana inmediatamente anterior, nivel=INCOMPLETO, indice conserva su número y motivo=HISTORIAL_INSUFICIENTE. Esto también se aplica si el índice actual es menor que 40.

2. Con semana anterior disponible: si indice actual <40, nivel=BAJO.

3. Con semana anterior disponible: si no se cumplen todas las condiciones de ALTO y el índice actual ≥40, nivel=MEDIO. Un pico ≥70 no sostenido se explica en el informe.

4. ALTO requiere simultáneamente: índice actual ≥70; índice anterior ≥70; y la intersección entre factores excedidos de ambas semanas contiene al menos dos códigos iguales (HE, DS, CT o VP). Se comparan índices numéricos, no el nivel previo. Por eso un primer período INCOMPLETO con 83,71 sí permite ALTO en el segundo.

- La semana anterior se calcula desde su RegistroLaboral usando las MISMAS reglas seleccionadas para la semana actual, incluso si su evaluación histórica usaba otra versión. Esta comparación se guarda en el snapshot; no se altera la evaluación publicada del período anterior. Un hueco de semanas no cuenta como persistencia.
- Importación fuera de orden: si llega la semana W después de W+1, procesar W y reevaluar W+1 si existe. La huella de W+1 incluye datos de W o su ausencia; debe cambiar aunque W+1 conserve sus propios datos. No hace falta recalcular W+2 por ese motivo: la persistencia depende de índices semanales, no del nivel anterior.

4.5 Configuración, versionado e idempotencia (RF05/RF06/RF09)

- ConfiguracionIndice v1: version='v1'; effectiveFrom='2026-01-05'; windowDays=7; persistencePeriods=2; weights HE/DS/CT/VP; thresholds HE/DS/CT/VP; mediumLimit=40; highLimit=70; minExceededFactors=2; createdBy y createdAt. En P0 la configuración se carga por seed/script, sin editor web.
- Validar pesos finitos entre 0,05 y 0,70 y suma 1 con tolerancia 0,000001; umbrales finitos >0 y CT≤1. P0 acepta exclusivamente windowDays=7 y persistencePeriods=2. Otras ventanas/persistencias se difieren y requieren ampliar contratos y pruebas.
- Validar límites: 0 < mediumLimit < highLimit ≤ 100. minExceededFactors entero entre 2 y 4, nunca mayor que el número de dimensiones activas. No aceptar configuración parcialmente guardada.
- Cada edición crea una versión inmutable con effectiveFrom en lunes y único. P0 permite nuevas reglas por script de Eros + Gonzalo: validar, persistir, crear trabajo de reevaluación para todas las semanas donde esa versión resulte aplicable y procesar en orden. Conservar resultados anteriores; marcar como vigente solo el último resultado correcto para cada par empleado/semana.
- Idempotencia: processingFingerprint = huella de registro actual + registro previo (o AUSENTE) + snapshot/versión de reglas + engineVersion='cu02-v1'. Clave única empleado+semana+processingFingerprint. Misma huella reutiliza evaluación; antecedentes nuevos o nueva regla crean otra. En transacción, supersedesEvaluationId enlaza la anterior y se actualiza el puntero vigente. Evitar que un job antiguo sustituya resultados de datos/reglas más nuevos mediante comprobación de versiones antes del commit.
- Ordenar ranking por contribución, pero mostrar también valor real, umbral y exceso. El ranking indica qué aportó MÁS al modelo, NO demuestra causalidad clínica ni que la variable causó burnout.

4.6 Escenarios que deben probarse

1. C1: primera semana válida, incluso con índice bajo → INCOMPLETO con número; dos semanas bajas consecutivas → segunda BAJO y sin alerta.
2. C2: índice entre 40 y 69,99 → MEDIO; sin alerta.
3. C3: con semana anterior existente, pico aislado ≥70 sin persistencia → MEDIO; sin semana anterior → INCOMPLETO. Sin al menos dos factores comunes excedidos, nunca ALTO.
4. C4: dos semanas consecutivas ≥ 70 con al menos dos mismas variables excedidas → ALTO; CU03.
5. C5: falta un obligatorio o hay A=0 con P/V>0 → CU01 rechaza el lote; llamada directa inválida al motor → error. Solo historial insuficiente produce

INCOMPLETO en P0; datos no procesados se muestran como PENDIENTE/FALLIDO.

6. C6: mismo fingerprint reutiliza evaluación; antecedentes tardíos o reglas nuevas crean evaluación vigente nueva sin borrar historial. C7: CT cero; Xi=Ui no excedida; índices 39,99/40/69,99/70 y desempates. Datos y resultados esperados en Contratos y pruebas.

5. CU03 · GENERAR INFORMES Y NOTIFICAR ALERTAS

5.1 Entradas y salidas

- Entrada desde CU02: toda Evaluacion vigente y persistida, sea BAJO, MEDIO, ALTO o INCOMPLETO, más empleado/equipo/responsable y snapshot de reglas. CU03 siempre genera informe; solo nivel=ALTO habilita alerta.
- Salida P0: InformeDTO para tablero y detalle, con índice, nivel, semana, ranking de cuatro factores, valores, umbrales, persistencia y recomendaciones. Exportación y vista consolidada avanzada pertenecen al sistema final.
- Salida alerta P0: Alerta persistida con evaluationId único, destinatario y vigencia, visible en el tablero local. No se envía correo. P1 añade notificador SIMULADO; el envío real queda para el sistema final.

5.2 Generación del informe

1. Transformar Evaluacion en un DTO de lectura: empleado, equipo, fecha/período, índice, nivel textual, scores, contribuciones y 3 o 4 factores ordenados.
2. Explicar las variables de manera operativa, por ejemplo «16 h extra frente al umbral configurado de 8 h»; no confundir contribución con prueba de causa.
3. Incluir observaciones: «dos semanas consecutivas», «historial insuficiente», «datos actualizados el...» y «versión de reglas utilizada». Mostrar INCOMPLETO explícitamente.
4. Resumen P0 por equipo/semana: contar únicamente evaluaciones vigentes y actualizadas, nunca históricas ni pendientes de reevaluación. noProcesados cuenta registros confirmados sin evaluación vigente actualizada. Porcentajes por nivel dividen por totalEvaluaciones, incluyendo INCOMPLETO; si total=0, porcentajes=0 y promedioIndice=null. Promedio excluye INCOMPLETO e informa muestraPromedio. El resumen no altera niveles individuales.
5. P0 no exporta archivos. En el sistema final, exportación de resultados con los mismos filtros y permisos de pantalla; empezar por CSV, ampliar después a XLSX/PDF. No incluir esta ampliación en la aceptación de T10/T14.

5.3 Condición de alerta, destinatarios y duplicados

1. Si el nivel no es ALTO, no crear alerta automática; el informe y el historial igualmente quedan disponibles.
2. Si es ALTO, buscar una alerta con evaluationId idéntico (clave única). Si ya existe, devolverla y NO reenviar correo automáticamente.

3. Destinatario P0 desde seed: técnico → responsable Team Lead del equipo; Team Lead → Gerente de Operaciones. Cada empleado evaluable tiene tipo TECNICO|TEAM_LEAD. Si el responsable falta o no corresponde, destinatarioId=null y motivo=SIN_DESTINATARIO; la alerta se conserva visible con incidencia, sin asignarla arbitrariamente a otra persona.
4. Guardar alerta con factores que superan umbral y sugerencias; canal=INTERNO, estadoEnvio=NO_APLICA y vigencia=ACTIVA. El informe conserva ranking completo. Si no hay destinatario, mostrar la incidencia junto a la alerta. P1 aplica visibilidad por rol.
5. Solo P1: simular el envío y guardar intento/resultado con canal SIMULADO. Si falla o falta destinatario, conservar alerta y registrar FALLIDO/SIN_DESTINATARIO. Reintentar el envío no crea otra alerta; ENVIADO debe verse como «Enviado (simulado)». Correo real y autenticación productiva quedan fuera del prototipo.
6. Si cambia la evaluación vigente del mismo empleado/semana, marcar SUPERADA la alerta anterior; nunca borrarla. Si la nueva es ALTO, crear su alerta única ACTIVA; si deja de ser ALTO, no crear otra. Otra semana ALTO tiene su propia alerta. Las consultas por defecto muestran vigentes/ACTIVAS y el detalle permite ver historial.

5.4 Recomendaciones, privacidad y aceptación

- Sugerencias NO automáticas: revisar asignación y vencimientos de tareas, redistribuir carga, evaluar descanso, evitar asignaciones adicionales. El sistema no ordena vacaciones ni altera turnos por sí mismo.
- Permisos objetivo (P1): RRHH importa y consulta todo; Team Lead ve solo técnicos de su equipo y sus alertas, sin ver índices de otros Team Leads; Gerencia ve consolidados y detalle/alertas de Team Leads; empleado no consulta índices ni alertas. En P0 existe solo operador demo local con todos los datos ficticios; no afirmar que hay autorización implementada.
- Sistema final: sesiones autenticadas, permisos en servidor, HTTPS, datos mínimos y trazabilidad. P1 comprueba controles de roles demo; no sustituye autenticación productiva. Un futuro correo real contendrá resumen mínimo y enlace autenticado. P0 se limita al entorno local ficticio de la matriz de alcance.
- Aceptación P0 CU03: todo nivel tiene informe; ALTO genera una alerta única por evaluationId; repetir no duplica; destinatario ausente deja incidencia visible; nueva evaluación supera alerta antigua y crea otra solo si ALTO. P1: el fallo/reintento simulado preserva la alerta.

6. DISEÑO DE LA PLANILLA UNIVERSAL (PROPUESTA)

6.1 Recomendación sobre «Actividad 1», «Actividad 2», «Actividad 3»

NO recomendar tres columnas fijas de actividades. Los nombres, cantidades y unidades dependen del rubro; habrá empresas con cero, tres o centenares de tareas. Además, tres cantidades de horas por actividad no permiten deducir tareas vencidas, asignadas o pendientes, que sí figuran en los requisitos y en la fórmula acordada.

Separar el tipo concreto de trabajo de la métrica universal. En lugar de «Actividad N (hs)», utilizar métricas agregadas de carga de trabajo: tareas activas, pendientes y vencidas. Cada organización puede mapear «tarea» a pedido, expediente, orden de trabajo, ticket o actividad, manteniendo definición consistente y fechas. Este mapeo requiere validación; no toda unidad es comparable entre organizaciones.

6.2 Hoja o CSV principal: Indicadores (una fila por empleado/semana)

Plantilla P0: CSV UTF-8 separado por comas; encabezados canónicos exactos, sin alias amistosos ni columnas desconocidas. Su orden puede variar; la plantilla sugerida usa el de abajo. El servidor los mapea a TypeScript mediante la tabla de Contratos y pruebas.

```
employee_id,period_start,period_end,horas_extra,dias_consecutivos_sin_descanso,tareas_activas,tareas_pendientes,tareas_vencidas,vacaciones_pendientes_dias,horas_trabajadas,dias_desde_ultimas_vacaciones
```

- Obligatorias: las primeras nueve columnas. Opcionales: horas_trabajadas y dias_desde_ultimas_vacaciones. Los datos auxiliares nunca sustituyen las cuatro dimensiones.
- CSV: ID texto; fechas YYYY-MM-DD; horas decimales con punto (8.5); días y tareas enteros; vacío solo en las dos columnas opcionales. Máximo 5 MiB y 1.000 filas. XLSX queda para el sistema final; no es formato de entrada P0/P1.
- Ejemplo de fila de prueba, NO datos reales: EMP-001,2026-10-12,2026-10-18,16,9,10,7,5,14,56,120. Para clasificar ALTO hace falta también una semana anterior válida que cumpla las condiciones.
- Evitar identificadores personales como DNI en la planilla. Usar códigos empleados opacos, estables y previamente asociados a un responsable/teamId en la base.

6.3 Datos detallados y cumplimiento integral de RF03

ADVERTENCIA DE ALCANCE: una fila agregada por semana hace posible el cálculo inicial, pero NO permite auditar por sí sola fechas de jornadas individuales, estado y vencimiento de cada tarea, ni fechas/estado de licencias exigidos en la redacción de RF03. Si se mantiene RF03 íntegro, deberán incorporar detalles o revisar formalmente ese requerimiento.

- Hoja opcional «Tareas»: task_id (string), employee_id (string), asignada_fecha (date), vencimiento_fecha (date), completada_fecha (date | null), estado (enum). Permite derivar activas, pendientes y vencidas al cierre de cada período sin columnas «Actividad N».
- Sistema final · Tareas: task_id, employee_id, asignada_fecha, vencimiento_fecha, completada_fecha nullable, estado. Para calcular el total semanal contar tareas abiertas al inicio o asignadas durante el período, incluidas las completadas dentro de él. Derivar pendientes y vencidas al cierre. Cancelaciones/reasignaciones requieren regla explícita en esa ampliación.
- Hoja opcional «Vacaciones»: employee_id, desde, hasta, estado, dias, saldo_pendiente; sin motivo médico ni datos de salud. Permite validar saldos y fechas.
- Un archivo XLSX puede tener varias hojas; si usan CSV, cada estructura detallada requiere su propio CSV. Para el primer prototipo puede usarse SOLO «Indicadores» como entrada consolidada, identificando esta simplificación en la entrega y en el alcance.

6.4 Cómo mantener la universalidad sin falsear el indicador

- El núcleo fijo analiza señales: horas extra, días consecutivos de trabajo, carga de pendientes/vencimientos y vacaciones pendientes. Las empresas NO deben editar el código para adaptarlas: cambian el origen, mapeo de datos y configuración versionada.
- Definir períodos y reglas dentro de cada empresa, no imponer umbrales legales absolutos válidos para todos los países o convenios. Los umbrales numéricos del documento son de DEMOSTRACIÓN y deben justificarse antes de uso real.
- No comparar directamente tareas de diferente esfuerzo solo por cantidad. Si una organización no gestiona tareas discretas, diseñar en otra versión un indicador alternativo medible, por ejemplo carga asignada/capacidad pactada; requiere recalibrar fórmula, reglas, pruebas y documentación.
- P0 exige las cuatro dimensiones: si faltan campos obligatorios, CU01 rechaza el archivo; CU02 invocado directamente devuelve error de contrato. Una organización sin métrica de tareas requiere otro esquema/modelo en una etapa posterior; no forzar INCOMPLETO ni redistribuir pesos para aceptarla.

7. ORGANIZACIÓN SUGERIDA EN NEXT.JS

- Interfaz: formulario de carga, previsualización, tabla de errores, tablero y detalle de informe. Todas las pantallas consumen respuestas tipadas y muestran nivel en texto + color.
- API P0 en Route Handlers de Next.js: validar, confirmar/cancelar, consultar job, reintentar job, listar evaluaciones, detalle y alertas. Las rutas y cuerpos exactos de «Contratos y pruebas» son obligatorios. El cliente renderiza; las reglas y persistencia se ejecutan en servidor.
- Servicios con firmas compartidas en Contratos y pruebas: validarImportacion → confirmarImportacion → procesarImportacion → calcularEvaluacion → construirInforme → crearAlertaSiCorresponde. Notificar se incorpora en P1. CU03 se invoca para todos los niveles, aunque solo ALTO cree alerta.
- Persistencia sugerida: empleados, equipos, usuarios/roles, importaciones, registros_laborales (con versiones), configuraciones_indice, evaluaciones, alertas, intentos_notificacion, auditoria.
- Índices únicos P0: registro por clave lógica+dataVersion y un puntero a versión vigente; evaluación por empleado+semana+processingFingerprint; alerta por evaluationId. ProcessingJob persistido permite reintentos. processingFingerprint incluye antecedente presente/ausente y reglas: no basta la huella actual.
- Procesamiento P0 acotado: confirmar persiste el job; el servidor intenta procesarlo tras el commit y responde al terminar o al primer fallo. No requiere broker/worker externo. Ante interrupción queda job recuperable; el endpoint de reintento lo reclama de forma exclusiva. Se documentan jobs detenidos y recuperación en Contratos y pruebas.

8. ACUERDOS Y CHECKLIST PARA EL EQUIPO

1. Aplicar frecuencia semanal y vocabulario de Objetivo y Modelo; no redefinir el total de tareas en cada módulo.
2. Publicar contratos exactos de la pestaña Contratos y pruebas como tipos compartidos en T01.
3. Usar reglas demostrativas v1 y ejemplos reproducibles de CU02; revisar su validez con la docente sin bloquear pruebas de integración.
4. P0 usa solo resúmenes CSV; RF03 completo (detalle de tareas/jornadas/vacaciones) sigue pendiente del sistema final y de su validación formal.
5. Respetar matriz P0/P1/final: roles demo y notificador simulado en P1; correo real y autenticación productiva, posteriores.
6. Repartir implementaciones sin ambigüedad: responsable de CU01 produce RegistroLaboral[] persistido; CU02 consume registros confirmados y devuelve Evaluacion; CU03 consume Evaluacion y gestiona informe + Alerta.

7. Probar escenarios de Contratos y pruebas: archivo inválido, duplicado, conflicto, primera semana numérica INCOMPLETA, segunda ALTO, fuera de orden y reintento. Roles/notificación simulada se verifican al implementar P1.
8. Actualizar «Grupo 8 - Entrega 3» antes de presentar: cambiar React+Python a Next.js+TypeScript si esta será la implementación; corregir o precisar RF03–RF09 según los acuerdos y la corrección docente.

9. FUENTES Y ESTADO DE VALIDACIÓN

Fuentes internas: «Grupo 8 - Entrega 3», particularmente propuesta de solución, RF03–RF12 y CU01–CU03; «8-Correccion 3», apartado «Entrega n°3 – Correcciones». Elaboración propia para los contratos, ejemplos, fórmula y diseño de planilla.

Estado: contrato operativo del prototipo; no aprobación docente ni validación clínica/laboral. HE=8 h, DS=6 días, CT=0,35, VP=10 días; pesos 30/30/25/15; cortes 40/70 y persistencia de dos semanas son parámetros demostrativos v1. Universalidad significa independencia de nombres de actividades, no calibración válida para cualquier rubro sin adaptación.

TAREAS DEL PRIMER PROTOTIPO

1. Tareas imprescindibles (P0)

T01 · Base del proyecto y contratos TypeScript

Responsable: Gonzalo · revisión Eros | Depende de: Ninguna

- **Hacer:** Inicializar Next.js App Router + TypeScript, scripts, .env.example y estructura compartida. Publicar todos los tipos, rutas y firmas de Contratos y pruebas, más mocks de sus respuestas. Eros revisa fórmulas/estados antes de integrar.
- **Listo cuando:** npm install, npm run build y pruebas base funcionan; todos importan los mismos tipos sin redefinirlos.

T02 · Persistencia Prisma/MySQL y datos semilla

Responsable: Gonzalo | Depende de: T01

- **Hacer:** Modelar empleados/equipos/responsables, importaciones, registros versionados, configuraciones, evaluaciones, alertas y ProcessingJob; claves únicas, vigentes e integridad. Migraciones, seed ficticio y comandos README. Incluir trazabilidad mínima P0; IntentoNotificacion y auditoría ampliada en P1.
- **Listo cuando:** La base se reconstruye desde cero y conserva empleados, registros, evaluaciones y alertas entre reinicios.

T03 · Plantilla de datos y archivos de prueba

Responsable: Santiago + Alfonso | Depende de: T01; T02 para IDs reales del seed

- **Hacer:** Crear templates/indicadores.csv con las nueve columnas obligatorias de Sugerencias y dos opcionales si se necesitan. Datos ficticios para 50 empleados y al menos dos semanas; incluir el ejemplo contractual y casos BAJO/MEDIO/ALTO, primera semana INCOMPLETA, hueco y carga fuera de orden. Archivos inválidos, idénticos y conflictivos. XLSX queda para el sistema final.
- **Listo cuando:** El CSV utiliza los mismos empleados y tipos que la base y permite reproducir los cuatro escenarios.

T09 · Configuración inicial y versionado de reglas

Responsable: Eros define · Gonzalo persiste | Depende de: T01, T02

- **Hacer:** Sembrar v1 efectiva desde 2026-01-05: HE=8, DS=6, CT=0,35, VP=10; pesos 0,30/0,30/0,25/0,15; límites 40/70 y dos semanas. Validar

configuración y habilitar nuevas versiones por script de Eros + Gonzalo con reevaluación; sin editor web.
- **Listo cuando:** Una regla inválida se rechaza; cada evaluación conserva el identificador de la versión aplicada.

T05 · Validaciones de importación y duplicados

Responsable: Santiago · revisión Eros | Depende de: T01, T03; T02 para cotejar registros

- **Hacer:** Validar campos obligatorios, empleado existente, semana/fechas reales, números no negativos y relación tareas_vencidas ≤ tareas_pendientes ≤ tareas_activas. Detectar duplicados internos, archivo previamente importado y conflicto por mismo empleado/período. Devolver error con fila/campo.
- **Listo cuando:** Un solo error impide confirmar todo el lote; la reimportación idéntica no duplica información.

T07 · Motor matemático CU02 y ranking

Responsable: Eros | Depende de: T01, T09 (definiciones acordadas)

- **Hacer:** Implementar calcularEvaluacion conforme a CU02, separando puntuación/clasificación del acceso a base. CT usa total semanal incluidas completadas; score_i=min(100,50×Xi/Ui); redondear solo índice final. Ranking por contribución con desempate HE/DS/CT/VP. Comparar el índice contractual redondeado.
- **Listo cuando:** Tests de normalización, valores límite, ranking, división por cero y ejemplo de índice ≈83,71; no requiere base para probar la fórmula.

T04 · API CU01 de validación y confirmación CSV

Responsable: Santiago · revisión Gonzalo | Depende de: T02, T03, T05

- **Hacer:** Implementar rutas de validación, confirmación/cancelación, job y reintento de Contratos y pruebas. Validar genera vista previa temporal; confirmar revalida y guarda en transacción; después solicita T08/T10/T11. Rechazar conflictos, no sobrescribir; procesar afectados y sucesor inmediato existente.
- **Listo cuando:** CSV válido se confirma una sola vez; CSV inválido no modifica la base; API entrega estado y cantidades.

T08 · Clasificación CU02, evaluación e historial

Responsable: Eros · apoyo Gonzalo | Depende de: T02, T07, T09

- **Hacer:** Clasificar según CU02 §4.4, conservando índice numérico en INCOMPLETO por falta de semana previa. La semana anterior se puntúa con

las mismas reglas actuales. Guardar snapshot y huella que incluye antecedentes, gestionar vigencia, historial, fuera de orden y reintentos sin duplicados. Fallos técnicos pertenecen al job.
- **Listo cuando:** Tests BAJO/MEDIO/ALTO/INCOMPLETO, pico aislado, misma evaluación repetida y cambio de versión sin perder historial.

T06 · Pantalla de carga y previsualización

Responsable: Alfonso + Francisco | Depende de: T01 y contrato de T04

- **Hacer:** Desarrollar interfaz Next.js para seleccionar CSV, Validar, mostrar vista previa, errores por fila y botones Confirmar/Cancelar. Contemplar carga, vacío y fallo. Conectar la vista al endpoint real; no calcular ni guardar directamente en navegador.
- **Listo cuando:** Se demuestra importación válida, inválida y repetida desde el navegador, sin herramientas externas.

T10 · Informe individual y explicación del índice

Responsable: Gonzalo | Depende de: T07, T08 y contratos T01

- **Hacer:** Preparar DTO de informe por empleado y semana con valor del índice, nivel, versión de reglas, valores y umbrales, ranking de contribuciones, condición de persistencia y sugerencias no automáticas. Tratar INCOMPLETO expresamente.
- **Listo cuando:** El detalle se obtiene de evaluaciones guardadas y es comprensible para el tablero, sin presentarse como diagnóstico de burnout.

T13 · API de consultas y resultados

Responsable: Gonzalo | Depende de: T02, T08, T10; T11 para mostrar alertas

- **Hacer:** Rutas GET de Contratos y pruebas para evaluaciones vigentes, detalle, resumen y alertas. Filtros/paginación y estados de proceso separados del nivel. T13 puede iniciar sin T11, pero se termina al consultar sus alertas reales. Roles demo se incorporan en T15a.
- **Listo cuando:** El frontend consume información persistida y actualizada; no usa arreglos precargados como resultados finales.

T11 · Generación de alertas internas

Responsable: Francisco · revisión Gonzalo | Depende de: T02, T08, T10; responsables del seed

- **Hacer:** Aplicar CU03: ALTO vigente crea una alerta única, canal INTERNO/NO_APLICA. Resolver técnico→Team Lead y Team Lead→Gerencia; ausencia deja incidencia. Al cambiar evaluación vigente

marcar alerta anterior SUPERADA y crear otra solo si el nuevo nivel es ALTO. El informe existe para todos los niveles.
- **Listo cuando:** Dos ejecuciones con el mismo evaluationId generan UNA alerta; queda persistida y visible. Gonzalo revisa las pruebas.

T14 · Tablero web con índices, factores y alertas

Responsable: Francisco · revisa Gonzalo | Depende de: T13; T11 para mostrar alertas

- **Hacer:** Crear vista por empleado/período/equipo con índice, texto BAJO/MEDIO/ALTO/INCOMPLETO, ranking, acceso al detalle y alertas. Puede comenzar con respuestas simuladas, pero debe conectar la API real; reducir gráficos/filtros si retrasan el circuito.
- **Listo cuando:** Tras importar un CSV, el tablero presenta los resultados almacenados y sus alertas, sin valores hardcodeados; Gonzalo revisa.

T16 · Integración integral y pruebas E2E

Responsable: Eros + Gonzalo coordinan · todos prueban | Depende de: T02, T04, T06, T08, T09, T10, T11, T13, T14

- **Hacer:** Integrar por contratos y pull requests: importar/validar CSV → guardar → evaluar → elaborar informe → generar alerta ALTO → mostrar tablero. Ejecutar escenarios válido, inválido, duplicado, historial insuficiente, ALTO sostenido y sin alerta duplicada. Resolver conflictos de integración y documentar ejecución.
- **Listo cuando:** Demo reproducible desde instalación limpia, con README, seed y datos ficticios. Cada integrante corrige su módulo.

2. Tareas complementarias (P1)

T12 · Simulador de correo y estados de envío

Responsable: Francisco + Alfonso | Depende de: T11

- **Hacer:** Notificador ficticio canal SIMULADO con intentos y estados PENDIENTE/ENVIADO/FALLIDO/SIN_DESTINATARIO. ENVIADO se muestra «Enviado (simulado)». Reintentar notificación sin recrear alerta. La alerta INTERNA P0 conserva NO_APLICA mientras no se simule.
- **Listo cuando:** Fallar o repetir un envío simulado no elimina ni duplica la alerta interna.

T15a · Usuarios de demostración y permisos

Responsable: Eros + Gonzalo | Depende de: T02, T04, T13

- **Hacer:** Preparar usuarios demo RRHH, Team Lead y Gerencia; aplicar controles de autorización EN SERVIDOR y limitar Team Lead a su equipo. Evitar acceso del empleado a índices/alertas. No usar esta demo como autenticación productiva.
- **Listo cuando:** Las pruebas bloquean un usuario sin permiso y la consulta de un equipo ajeno.

T15b · Auditoría de importaciones y reglas

Responsable: Francisco + Santiago | Depende de: T02, T04, T09

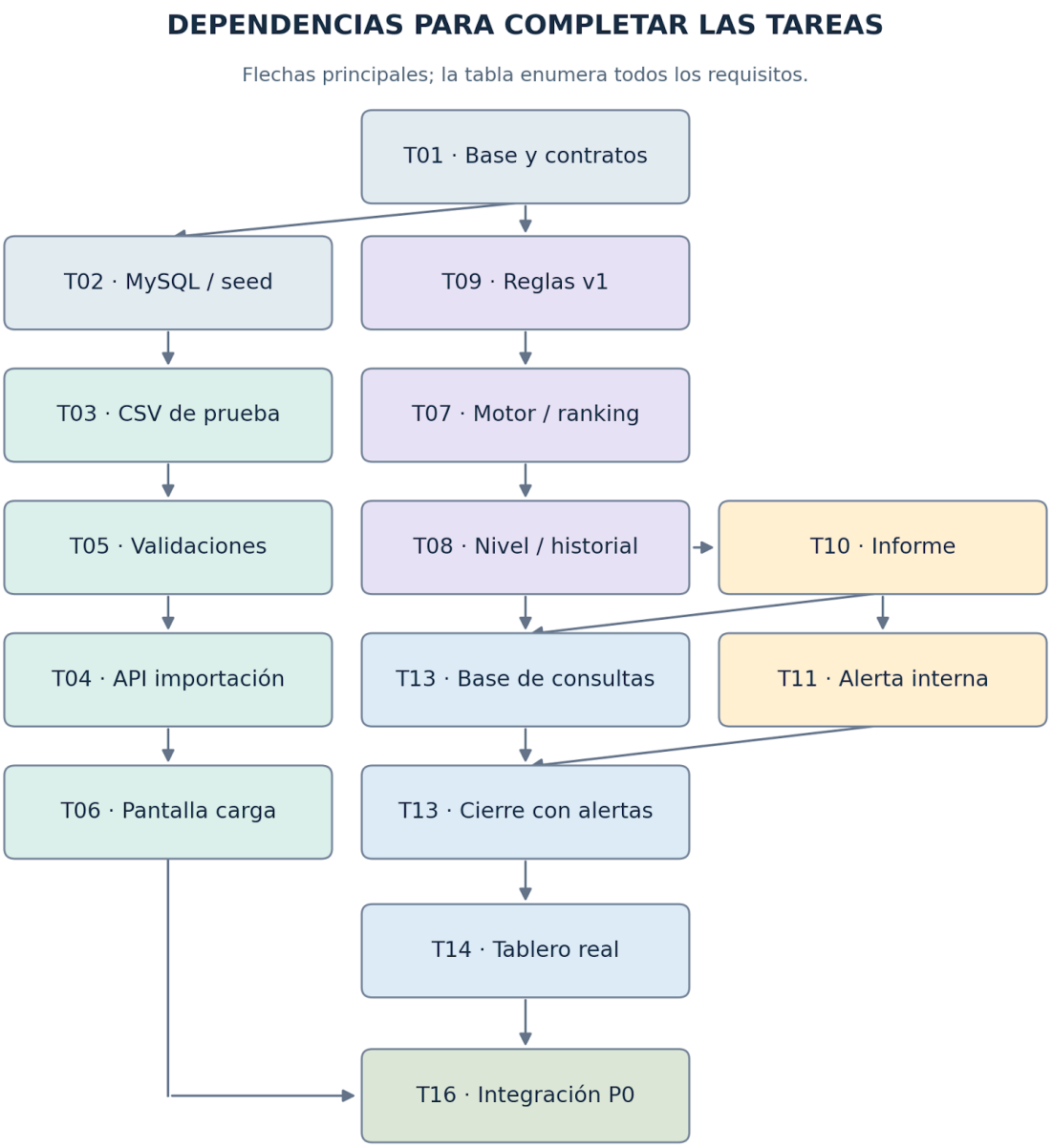
- **Hacer:** Ampliar eventos y consultas de auditoría para validaciones rechazadas, cancelación, confirmación y reglas/versiones/usuario. Los metadatos de carga, snapshots y versiones ya son P0; T15b no los posterga. Evitar contraseñas y datos innecesarios en logs.
- **Listo cuando:** Se rastrean importaciones aceptadas/rechazadas y cambios de reglas sin borrar trazabilidad anterior.

3. Tabla de tareas, responsables y dependencias

Orden	Tarea	Responsable	Depende de
01 · P0	T01 Base y contratos	Gonzalo · revisa Eros	Ninguna
02 · P0	T02 Prisma/MySQL	Gonzalo	T01
03 · P0	T03 Plantilla/dataset	Santiago + Alfonso	T01; T02 IDs
04 · P0	T09 Reglas versionadas	Eros define; Gonzalo guarda	T01, T02
05 · P0	T05 Validaciones RF04	Santiago · revisa Eros	T01, T03, T02
06 · P0	T07 Fórmula + ranking	Eros	T01, T09
07 · P0	T04 API de importación	Santiago · revisa Gonzalo	T01, T02, T03, T05
08 · P0	T08 Clasificación/historial	Eros · apoya Gonzalo	T02, T07, T09
09 · P0	T06 Pantalla de carga	Alfonso + Francisco	T01, T04
10 · P0	T10 Informe individual	Gonzalo	T01, T08

11 · P0	T13 API de consultas	Gonzalo	T02, T08, T10; T11 alertas
12 · P0	T11 Alertas internas	Francisco · revisa Gonzalo	T02, T08, T10
13 · P0	T14 Tablero web	Francisco · revisa Gonzalo	T01, T13 completo
14 · P0	T16 Integración y pruebas	Eros + Gonzalo; todos prueban	Todas las P0: T01–T11, T13, T14
P1	T12 Correo simulado	Francisco + Alfonso	T11
P1	T15a Permisos	Eros + Gonzalo	T02, T04, T13
P1	T15b Auditoría	Francisco + Santiago	T02, T04, T09

4. Mapa de tareas y dependencias (P0)



PLAN DE TAREAS · PRIMER PROTOTIPO

Grupo 8 · Next.js + TypeScript · 5 integrantes · Documento operativo

Objetivo P0: prototipo web local con datos ficticios. CSV → CU01 valida/confirma → CU02 calcula por empleado/semana → CU03 genera informe y alerta interna cuando corresponda → tablero. Alcance exacto: matriz en Objetivo y Modelo. Contratos, estados y ejemplos: Contratos y pruebas.

Asignaciones vigentes indicadas por Eros en este documento. Tareas Resumen permite consulta rápida; esta pestaña desarrolla entregables y conserva un cronograma orientativo. Los tipos/API y fórmulas se toman de Contratos y pruebas y CU02. Trello es una herramienta de seguimiento externa; esta edición no actualiza sus tarjetas.

1. DEFINICIÓN DE TERMINADO (PRIMER PROTOTIPO P0)

- Una aplicación Next.js + TypeScript se inicia localmente siguiendo README; base MySQL + Prisma migrada y poblada con datos ficticios.
- La primera demo debe admitir CSV de empleados semanales, mostrar errores por fila, confirmar lotes válidos y observar resultados. XLSX es una ampliación compatible que puede completarse después del circuito CSV.
- Ningún lote inválido deja datos parciales. La misma importación repetida no duplica registros ni evaluaciones.
- CU02 procesa las cuatro dimensiones acordadas, calcula índice 0–100, ranking, nivel BAJO/MEDIO/ALTO/INCOMPLETO y conserva regla/versiones.
- CU03 genera informe individual; si ALTO es sostenido, crea una sola alerta interna con destinatario. La simulación de envío de correo es P1: no bloquea el primer recorrido.
- P0: vista web básica por empleado/período y estado de procesamiento; datos ficticios en entorno local. Trazabilidad mínima y snapshots son P0; usuarios/roles demo y auditoría ampliada son P1. La demo no implementa autenticación productiva.
- Se ejecuta una demo completa de 50 empleados simulados y dos semanas o más, acompañada de pruebas automatizadas y manuales.

2. RESPONSABILIDADES VIGENTES

REPARTO ACTUALIZADO: Eros lidera el proyecto, define las reglas y desarrolla CU02 (T07/T08/T09). Gonzalo lidera arquitectura y persistencia, configura Next.js/Prisma, implementa informe CU03 (T10), API de consultas (T13) e integración (T01/T02/T16). Santiago desarrolla plantilla, validaciones e importación CU01 (T03/T05/T04). Alfonso comparte con Francisco la pantalla de carga (T06) y el simulador de correo (T12, P1). Francisco implementa alertas internas (T11), el

tablero (T14; revisa Gonzalo) y comparte T06 con Alfonso. T15 se divide: permisos T15a = Eros + Gonzalo; auditoría T15b = Francisco + Santiago.

Regla de gestión: cada tarea tiene entregable verificable, dependencia explícita y revisión. T11 queda a cargo de Francisco y, por ser fundamental para CU03, Gonzalo revisa sus tests de duplicados y persistencia antes de integrar. T14 queda a cargo de Francisco con revisión obligatoria de Gonzalo; T06 y T12 son trabajos compartidos. Esta asignación equilibra cantidad, pero no necesariamente dificultad: Gonzalo todavía concentra varias dependencias críticas. Todos deben comunicar bloqueos y cumplir los criterios de aceptación.

3. PRIMERA ETAPA · BLOQUEANTES (ANTES DE PROGRAMAR EN PARALELO)

T01 · Base Next.js y contratos TypeScript (Gonzalo; Eros aprueba)

- Inicializar proyecto App Router, TypeScript, estructura de carpetas, npm scripts, pruebas y .env.example sin credenciales.
- Publicar tipos RegistroLaboral, Evaluacion, ConfiguracionIndice, ResultadoImportacion y Alerta, y documentación de flujo y errores.
- Entrega: repositorio ejecutable y archivos compartidos importables por todos. Aceptación: npm install/build/test OK y todos usan el mismo contrato.

T02 · Prisma/MySQL: esquema, migraciones, seed (Gonzalo)

- Crear modelos Empleado, Equipo, Importacion, RegistroLaboral versionado, ConfiguracionIndice, Evaluacion, Alerta y ProcessingJob; punteros vigentes, snapshots y trazabilidad mínima P0. Usuarios/roles, IntentoNotificacion y eventos ampliados se incorporan en P1.
- Definir referencias, integridad y claves únicas; migrar y poblar empleados, equipos y responsables ficticios.
- Entrega: Prisma schema, migrations, seed y comandos README. Aceptación: otro integrante instala la base desde cero.

T03 · Planilla universal + datos ficticios (Santiago + Alfonso)

- Entregar primero templates/indicadores.csv (XLSX como ampliación) con encabezados: employee_id, period_start, period_end, horas_extra, dias_consecutivos_sin_descanso, tareas_activas, tareas_pendientes, tareas_vencidas, vacaciones_pendientes_dias; opcionales horas_trabajadas, dias_desde_ultimas_vacaciones.
- Generar 50 empleados ficticios con dos semanas o más y casos BAJO/MEDIO/ALTO, primera semana INCOMPLETA, huecos y fuera de

orden. Incluir ejemplo y verificaciones de Contratos y pruebas; archivos inválidos, idénticos y conflictivos por separado.
- **Aceptación:** la planilla coincide EXACTAMENTE con el contrato de CU01 y con el seed de empleados.

4. CU01 · IMPORTACIÓN Y PANTALLA DE CARGA

T04 · API de ingesta CSV (Santiago; revisión Gonzalo)

- Implementar rutas de Contratos y pruebas: validar CSV, confirmar/cancelar y consultar/reintentar job. Parsear encabezados exactos y construir entrada normalizada. XLSX no es obligatorio P0/P1.
- Validar conserva solo vista previa temporal. Confirmar revalida contra la base y guarda registros+job en transacción. Después procesa pares afectados y su semana siguiente existente; fallos posteriores no deshacen el commit ni crean datos parciales.
- Aceptación P0: CSV válido genera datos normalizados y confirmar devuelve importId y conteos. Prueba adicional XLSX al implementar ese formato.

T05 · Validación RF04 + idempotencia (Santiago; revisión Eros)

- Rechazar faltantes, ID inexistente, fechas/semana inválidas, negativos, no finitos y todo caso que viole 0≤vencidas≤pendientes≤total. Aplicar límites de CU01; total incluye tareas completadas durante la semana.
- Detectar duplicado en archivo, importación idéntica histórica y conflicto mismo empleado/período con datos diferentes; retornar fila, columna y detalle.
- **Aceptación:** lote con una sola fila inválida no modifica NADA; repetir importación idéntica no duplica registros.

T06 · Pantalla de carga y previsualización (Alfonso + Francisco)

- Selector solo .csv en P0; botones Validar/Confirmar/Cancelar, resumen, tabla de errores y estados de carga/éxito/fallo. Mostrar confirmación separada de procesamiento; job FALLIDO ofrece reintento, nunca otra importación.
- Conectar a API REAL; no realizar cálculo ni persistencia en el navegador. El resultado debe informar empleados/períodos procesados.
- **Aceptación:** se puede demostrar archivo válido, inválido y repetido desde la interfaz sin usar Postman.

5. CU02 · CÁLCULO (EROS)

T07 · Motor de puntuación + ranking (Eros)

- Implementar cuatro variables HE, DS, CT y VP, normalización score=min(100,50×valor/umbral) y pesos iniciales 0,30/0,30/0,25/0,15.
- CT=0,60×(tareasVencidas/tareasActivas)+0,40×(tareasPendientes/tareasActivas). Con activas=0, CT=0 solo si pendientes=vencidas=0.
- Devolver índice 0–100, scores y contribuciones ponderadas ordenadas. No confundir ranking de contribuciones con causas clínicas.
- **Aceptación:** caso numérico de Base Teorica reproduce índice aproximado 83,71; sin NaN ni divisiones por cero.

T08 · Clasificación, persistencia e historial (Eros; apoyo Gonzalo)

- Aplicar CU02 §4.4: primera semana sin antecedente → INCOMPLETO con indice numérico; con anterior válida, BAJO <40, ALTO requiere dos índices ≥70 y dos mismos factores excedidos; resto ≥40 → MEDIO. Comparar la semana anterior bajo las reglas de la actual.
- Guardar snapshots, antecedentes presentes/ausentes, processingFingerprint y reglas; mantener versión vigente e historial. Reprocesar sucesor inmediato al incorporar antecedentes tardíos; misma huella reutiliza evaluación. Error técnico marca job FALLIDO, no INCOMPLETO.
- **Aceptación:** pruebas de las cuatro categorías, persistencia, pico aislado y recalculo inmutable.

6. REGLAS COMPARTIDAS, INFORMES Y ALERTAS

T09 · Reglas v1 y versiones (Eros define; Gonzalo implementa persistencia)

- Sembrar v1 efectiva desde 2026-01-05: HE=8 h, DS=6 días, CT=0,35, VP=10 días; pesos 0,30/0,30/0,25/0,15; cortes 40/70; ventana 7 días y persistencia 2 semanas. Cambios por script crean versión inmutable y job de reevaluación.
- Validar coherencia, ventana semanal, límites y nueva versión por modificación; no reemplazar configuraciones históricas.
- **Aceptación:** configuración inválida se rechaza y Evaluacion conserva rulesVersion.

T10 · Informe individual y ranking explicable (Gonzalo)

- Armar detalle por empleado con período, índice, nivel, factores ordenados, valores reales y umbrales, explicación de persistencia y sugerencias.

- Incluir INCOMPLETO y fecha/version de reglas; fuente real Evaluacion persistida.
- **Aceptación:** informe consumible desde API/tablero en los cuatro estados; sin afirmar diagnóstico de burnout.

T11 · Crear alerta interna una sola vez (Francisco)

- Solo ALTO vigente crea alerta, canal INTERNO y estadoEnvio NO_APLICA. Técnico→Team Lead; Team Lead→Gerencia. Sin destinatario: conservar incidencia. Clave única evaluationId; Gonzalo revisa.
- Registrar factores excedidos y sugerencias. Si una evaluación nueva sustituye a otra en la misma semana, marcar su alerta SUPERADA. Crear una alerta nueva solo si el resultado nuevo es ALTO; conservar historial y no reenviar por refrescos.
- **Aceptación:** dos llamadas de igual evaluationId generan una alerta, visibles para RRHH/responsable.

T12 · Simulador de correo y errores · P1 (Francisco + Alfonso)

- P1: canal SIMULADO con estados ENVIADO/FALLIDO/PENDIENTE/SIN_DESTINATARIO y registro de intentos. Mostrar «Enviado (simulado)». P0 usa NO_APLICA sin intentar correo.
- P1: el correo simulado puede desarrollarse después de integrar P0; el correo REAL queda para después del MVP. Nunca presentar simulación como envío real.
- **Aceptación:** falla simulada no elimina alerta y reintento no la duplica.

7. PANTALLAS, API E INTEGRACIÓN (GONZALO + ALFONSO + FRANCISCO)

T13 · API de lectura de evaluaciones y alertas (Gonzalo)

- Implementar GET exactos de Contratos y pruebas: listado paginado, detalle, resumen y alertas vigentes; separar versiones históricas, vigencia y estados de procesamiento.
- P0 opera con identidad demo local; P1 añade autorización en servidor y filtros por ámbito. T13 se cierra con lectura real de T11; puede iniciarse antes con mocks del mismo contrato.
- Aceptación P0: Francisco renderiza la respuesta real en T14, incluyendo alertas. Aceptación P1: Team Lead no consulta equipos ajenos.

T14 · Tablero web y detalle (Francisco; revisa Gonzalo)

- Tabla por empleado, filtros, tarjetas de nivel, detalle con ranking y alertas internas, estados vacíos, fechas y reglas.

- Mostrar niveles con texto más color, INCOMPLETO explícito, ningún resultado hardcodeado.
- **Aceptación:** interfaz usa la misma evaluación que CU02 guardó; se ve cambio tras importar archivo.

T15 · Permisos (Eros + Gonzalo) y auditoría (Francisco + Santiago) · P1

- Precargar RRHH, Team Lead y Gerencia; asegurar en servidor permisos por rol/ámbito, sin considerar el prototipo como login productivo.
- Registrar operaciones de importación y cambios de reglas con autor y resultado; logs sin información sensible.
- Aceptación P1: consulta de equipo ajeno rechazada y eventos ampliados consultables; la trazabilidad mínima de cada importación confirmada ya debe existir en P0.

T16 · Integración final y pruebas (Eros + Gonzalo dirigen; todos prueban)

- Fusionar PR por orden contratos → DB → CU01 → CU02 → CU03 → API → pantallas. Revisión técnica cruzada y build/test tras cada integración.
- Pruebas P0: escenarios de Contratos y pruebas, incluyendo fuera de orden y recuperación de job fallido; ejemplo integral de dos semanas, versión vigente y alerta única. P1: notificación simulada, permisos y auditoría ampliada. XLSX se prueba en el sistema final.
- Entregar README reproducible, capturas o demo en vivo, lista de limitaciones y pendientes. Criterio: demo ejecutada desde instalación limpia y datos ficticios.

8. ORDEN DE EJECUCIÓN Y DEPENDENCIAS

1. Fase A · Todos acuerdan contratos y alcances. Gonzalo configura Next.js/Prisma y publica modelos; Eros define y valida el cálculo; Santiago prepara la plantilla; Alfonso y Francisco acuerdan las interfaces de T06. Nadie crea campos distintos a los contratos compartidos.
2. Fase B · Trabajo paralelo: Santiago implementa CU01 servidor y validaciones; Eros implementa CU02 con pruebas; Gonzalo integra persistencia, reglas, informe CU03 y API de consulta; Alfonso + Francisco preparan T06; Francisco implementa después T11 y T14, con revisión de Gonzalo. Lo que no dependa de datos reales puede desarrollarse con mocks.
3. Fase C · Eros y Gonzalo dirigen integración, merges y pruebas. Santiago corrige CU01. Alfonso y Francisco completan y conectan las pantallas. Gonzalo revisa expresamente T11 y T14. Evitar que la integración descubra incompatibilidades de nombres o tipos por primera vez.

4. Fase D · E2E con dataset real del prototipo, revisar PR, corregir incidencias y preparar recorrido docente.

9. TAREAS POSTERIORES AL PRIMER PROTOTIPO

- Correo REAL por SMTP/API, con secretos, reintentos controlados y comprobación de destinatario.
- Interfaz de edición de pesos/umbrales y versiones para RRHH (RF05 completo).
- Exportación CSV/XLSX de reportes (RF11), CRUD de empleados/equipos/roles (RF01/RF02) y vista propia del empleado (RF10).
- Registros desagregados por jornada/tarea/licencia, si se exige RF03 íntegro. Una planilla semanal resumida NO permite comprobar todas esas fechas y estados.
- Autenticación y autorización productivas, despliegue, monitoreo, backups y pruebas de seguridad/rendimiento más amplias.

10. GESTIÓN EN TRELLO Y CRITERIOS DE TRABAJO

Este plan contiene 14 tareas P0, 3 P1 y ampliaciones posteriores. Tareas Resumen y esta pestaña deben conservar responsables y dependencias idénticos. Trello puede quedar desactualizado hasta sincronizarlo; para implementar prevalece este documento.

- Mover una tarjeta a En proceso solo cuando haya responsable real confirmado y se hayan resuelto sus dependencias. Al terminar, adjuntar PR o commit y capturas o resultado de tests antes de Hecho.
- Los responsables de esta versión son los indicados por Eros. Para asignación nativa en Trello, comprobar primero los miembros actuales; la composición del tablero no se da por verificada en esta edición.
- Las tarjetas de CU01, CU02 y CU03 deben respetar exactamente las pestañas técnicas existentes. Si cambia un campo/umbral, modificar primero el contrato acordado y avisar al equipo.
- No establecer fechas arbitrarias: el Plan antiguo corresponde a un meet anterior y menciona Python; usar como referencia histórica, no como cronograma vigente.

Reglas v1 fijas para programar/probar el prototipo; pendientes de validación del indicador con la profesora. Nuevos acuerdos deben actualizar fórmula, contratos y pruebas antes de modificar código que consumen otros módulos.

11. PRIORIDADES, ORDEN Y DEPENDENCIAS EXACTAS

P0 = imprescindible para demostrar el flujo completo con CSV, datos ficticios, persistencia, cálculo, informe y alerta interna. P1 = complementario: simulación de correo, autenticación/roles demo y auditoría ampliada. Son 14 tarjetas P0 y 3 tarjetas P1. Las 5 tarjetas «POST-MVP» siguen fuera de este corte.

11.1 Orden de ejecución recomendado (respetando dependencias)

PASO 1 · T01 (Gonzalo, aprobación Eros): preparar Next.js, estructura del repositorio, contratos TypeScript, scripts y errores comunes. Es la base de las demás tareas.

PASO 2 · En paralelo y con revisión: T02 (Gonzalo, Prisma/seed), T03 (Santiago + Alfonso, plantilla y datos simulados) y T09 (Eros define reglas; Gonzalo versiona). El dataset se cierra con IDs de T02.

PASO 3 · En paralelo: T05 (Santiago; revisa Eros, validación) y T07 (Eros, fórmula pura y ranking). T07 no requiere que CU01 ni la base de datos estén implementados para sus pruebas unitarias.

PASO 4 · En paralelo: T04 (Santiago; revisa Gonzalo, API de validación/confirmación) y T08 (Eros; Gonzalo apoya persistencia, clasificación/historial). Ambos consumen los contratos compartidos.

PASO 5 · En paralelo: T06 (Alfonso + Francisco, interfaz de carga), T10 (Gonzalo, informe explicable) y T13 (Gonzalo, API de consultas). T06 puede empezar con una respuesta simulada y se cierra conectándose con T04.

PASO 6 · En paralelo: T11 (Francisco, alerta interna única; revisa Gonzalo) y T14 (Francisco, tablero; revisa Gonzalo). T11 necesita una evaluación ALTO guardada; T14 necesita T13. Como ambas recaen sobre Francisco, priorizar T11 y dejar la parte visual mínima para después.

PASO 7 · T16 (Eros y Gonzalo coordinan): integración, PR, pruebas E2E y demostración CSV → CU01 → CU02 → CU03 → tablero. Santiago corrige CU01; Alfonso y Francisco conectan T06; Francisco completa T11/T14; Gonzalo valida sus resultados.

PASO 8 · Solo cuando funcione P0: T12 (Francisco + Alfonso, simulador de correo), T15a (Eros + Gonzalo, permisos) y T15b (Francisco + Santiago, auditoría). XLSX, gráficos y filtros complejos tampoco bloquean la primera demostración.

11.2 Dependencias por tarjeta (contrato de trabajo)

T01 · Base/contratos [Gonzalo, aprueba Eros]: sin dependencias. Primera entrega.

T02 · Modelos/Prisma/seed [Gonzalo]: T01.

T03 · Plantilla + dataset CSV [Santiago + Alfonso]: T01; T02 para IDs de seed consistentes.

T09 · Reglas v1 [Eros define; Gonzalo guarda/versiona]: T01, T02.

T05 · Validación RF04 [Santiago, revisa Eros]: T01, T03 y T02 para cotejar duplicados históricos.

T07 · Puntuación y ranking [Eros]: T01 y definición de T09; sin dependencia de BD para unit tests.

T04 · Ingesta/confirmación [Santiago, revisa Gonzalo]: T01, T02, T03, T05.

T08 · Clasificación e historial [Eros, apoya Gonzalo]: T07, T09; T02 para almacenar resultados.

T06 · UI de importación [Alfonso + Francisco]: T01 y contrato de T04. Primero mocks; antes de Hecho debe usar el endpoint real.

T10 · Informe explicable [Gonzalo]: T07, T08, DTO de T01.

T11 · Alerta interna [Francisco; Gonzalo revisa]: T02, T08, T10 y responsables precargados. Pruebas obligatorias: solo ALTO, sin duplicados.

T13 · API de consultas [Gonzalo]: T01, T02, T08, T10; incluir T11 para alertas.

T14 · Tablero [Francisco; revisa Gonzalo]: T01 y T13 completo (incluye T11 para alertas). Puede empezar con mocks; Hecho exige resultados reales y estados de procesamiento.

T16 · Integración E2E [Eros + Gonzalo; todos apoyan]: T04, T06, T08, T10, T11, T13, T14; también T02 y T09. No depende de P1.

T12 [Francisco + Alfonso, P1]: depende de T11. T15a [Eros + Gonzalo, P1]: depende de T02, T04 y T13. T15b [Francisco + Santiago, P1]: depende de T02, T04 y T09.

11.3 Acuerdo de integración

Crear rama por tarea, abrir PR y aprobar criterios de aceptación antes de mover tarjeta a Hecho. No fusionar código que cambie unilateralmente los tipos compartidos.

Eros coordina el equipo, los criterios de CU02 y las decisiones de alcance. Gonzalo dirige la arquitectura e integración y revisa T11 y T14. Santiago implementa CU01; Alfonso y Francisco comparten T06. Francisco implementa T11 y T14, y después

participa en T12/T15b. Cada autor entrega tests y evidencia; una tarea compartida no significa que ninguno tenga responsabilidad.

No interpretar «orden» como trabajo estrictamente secuencial: muchas tareas avanzan en paralelo con mocks, pero ninguna se declara terminada mientras falten dependencias reales.

12. ¿BASE DE DATOS PARA EL PRIMER PROTOTIPO?

La base es obligatoria en este P0 integrado: Prisma + MySQL conserva historial, idempotencia, trabajos recuperables y alertas. Solo el motor matemático se prueba sin base, como función pura; esta separación no cambia el alcance persistente de la demo.

Esquema mínimo para P0: empleados/equipos/responsables; registros semanales e importaciones; reglas v1; evaluaciones versionadas; alertas únicas. Puede diferirse el modelo de login/auditoría avanzada hasta P1; si se conserva por comodidad en Prisma, no requiere pantalla ni flujo completo en P0.

Separación fundamental: la función pura CU02 recibe un RegistroLaboral y sus antecedentes/reglas como argumentos, calcula y devuelve un resultado. Otro servicio obtiene y guarda esos datos con Prisma. Así Eros puede probar CU02 desde el primer día aunque Gonzalo todavía esté migrando la base.

No usar información laboral real sin implementar primero autenticación, permisos, controles de acceso y protección de datos. El P0 simplificado es exclusivamente una demostración local con datos simulados.

13. DEFINICIÓN DE ALCANCE CON LA PROFESORA

Las 14 tareas P0 forman el primer prototipo integrado; no deben confundirse con el alcance final de RF01–RF12. Antes de presentar, declarar expresamente: datos agregados semanales; CSV obligatorio y XLSX posterior; correo no real; configuración v1 de ejemplo; historial y alerta interna funcionales. Pedir validación de esta simplificación si la entrega exige ya todas las funcionalidades

14. TABLA EJECUTIVA DE IMPLEMENTACIÓN (P0)

Las fases orientan el inicio, no son una cola estricta. Los mocks permiten avanzar interfaces sin cerrar backend; terminar requiere dependencias reales. T13 inicia con evaluaciones/informes y se cierra al integrar alertas T11; T14 se cierra con T13 completo.

Tabla 1. Ruta de trabajo del prototipo y dependencias directas

.

Fase	Tarea P0	Responsable / revisión	Depende de
1	T01 · Base y contratos	Gonzalo · revisa Eros	Inicio
2	T02 · Prisma y seed	Gonzalo	T01
2	T03 · Planilla + dataset	Santiago + Alfonso	T01, T02 (IDs)
2	T09 · Reglas v1	Eros + Gonzalo	T01, T02
3	T05 · Validaciones	Santiago · revisa Eros	T01, T03, T02
3	T07 · Fórmula y ranking	Eros	T01, T09
4	T04 · API de ingesta	Santiago · revisa Gonzalo	T02, T03, T05
4	T08 · Clasificación	Eros · apoya Gonzalo	T02, T07, T09
5	T06 · Pantalla de carga	Alfonso + Francisco	T01, T04
5	T10 · Informe CU03	Gonzalo	T07, T08
5	T13 · API de consultas	Gonzalo	T01, T02, T08, T10; T11 para cierre
6	T11 · Alerta interna	Francisco	T02, T08, T10
6	T14 · Tablero web	Francisco · revisa Gonzalo	T01, T13 completo
7	T16 · Integración / pruebas	Eros + Gonzalo · todos	T04, T06, T08, T10, T11, T13, T14

15. MAPA VISUAL: ORDEN Y DEPENDENCIAS

Mapa de desarrollo: las flechas muestran requisitos para completar entregables. Se distinguen T13 base y su cierre con alertas para evitar una falsa dependencia circular. «Pares afectados» y ejecución automática pertenecen al flujo del sistema, mostrado aparte en Contratos y pruebas. Ningún diagrama incluye responsables.

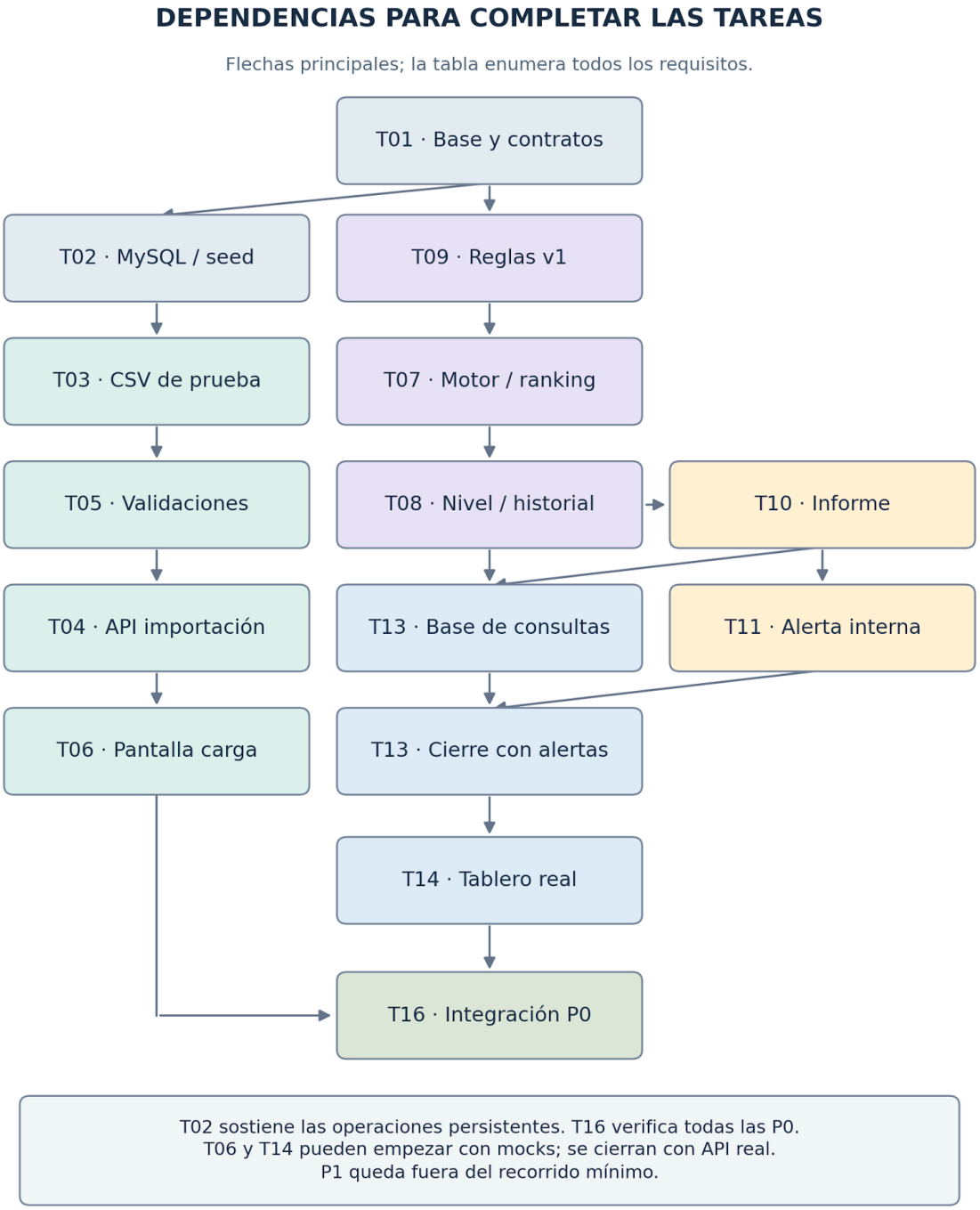


Figura 1. Dependencias para completar tareas P0; los requisitos secundarios figuran en la tabla.

16. ACUERDOS DE ASIGNACIÓN Y CONTROL

• P0 (obligatorio): T01, T02, T03, T04, T05, T06, T07, T08, T09, T10, T11, T13, T14 y T16. No se declara «Hecho» sin prueba funcional y revisión de su responsable principal.

• P1 (no bloqueante): T12 simulador de correo [Francisco + Alfonso]; T15a permisos [Eros + Gonzalo]; T15b auditoría [Francisco + Santiago]. Los controles de seguridad deben completarse antes de utilizar datos laborales reales.

• Los nombres designan responsables de implementación, acompañamiento y revisión. Cada tarea debe tener un resultado verificable. T11 requiere revisión técnica por Gonzalo, aunque Francisco sea el único responsable de implementarla; T14 requiere revisión por Gonzalo.

• Evitar dependencias humanas únicas: los contratos y pruebas deben poder ejecutarse por cualquier integrante. El repositorio y las tareas de Trello deben sincronizar sus asignaciones con este documento antes del comienzo del trabajo.

17. VIABILIDAD EN CINCO DÍAS Y EQUILIBRIO DEL EQUIPO

Evaluación: el reparto NO está equilibrado por complejidad, aunque todos tengan tareas. Gonzalo concentra configuración base, persistencia, informe, API y revisión de tareas ajenas. Francisco tiene T06 compartida y dos tareas P0 consecutivas y críticas (T11, T14). Si T11 se atrasa, afecta alertas y demostración; si T14 se atrasa, afecta la visualización. Santiago concentra CU01; Eros concentra CU02; Alfonso tiene participación acotada. El riesgo del cronograma depende de la disponibilidad real de todos.

Cinco días es una meta POSIBLE pero exigente para un prototipo local, no una garantía. El equipo debe reservar dedicación efectiva, trabajar en paralelo e integrar diariamente. No prometer completar RF01–RF12. Alcance máximo sugerido: CSV, planilla semanal, reglas v1, índice + ranking, alerta interna única, informe/tablero simple, base persistente y datos 100 % ficticios. Excluir XLSX si demora, correo real, pantallas complejas, CRUD y login productivo.

Plan mínimo de cinco jornadas (con criterio de salida)

- DÍA 1 · T01 y contratos cerrados; T02 comienza con migración/seed; T03 entrega CSV alineado; T09 define regla v1. Salida: proyecto ejecutable, objetos TypeScript comunes, datos simulados de prueba.
- DÍA 2 · T05 validaciones y avance de T04; T07 índice probado con datos en memoria. T06 puede montarse con un mock. Salida: importar/validar un CSV y obtener valores CU02 consistentes en pruebas unitarias.

- DÍA 3 · Cerrar T04 y T08 con persistencia; Gonzalo publica DTO T10 y ruta T13 inicial. Francisco empieza T11 con un caso ALTO precargado. Salida: flujo servidor CSV → registro → evaluación → alerta en la base, aunque la UI no esté completa.
- DÍA 4 · Alfonso/Francisco completan T06; Francisco desarrolla T14 con datos reales; Gonzalo revisa T11 y T14. Eros/Gonzalo hacen primera prueba integrada. Salida: demostración funcional de principio a fin, sin depender de pantallas sofisticadas.
- DÍA 5 · T16: corregir errores, validar datos incorrectos/duplicados, persistencia de dos semanas, alerta única, índices y reporte; actualizar README y ensayar demo. Reserva de contingencia: NO iniciar P1 hasta que P0 funcione.

Regla de corte: si al cierre del DÍA 3 no existe un circuito servidor CU01→CU02→CU03 con un caso ALTO, simplificar la interfaz a una lista/tabla mínima y suspender todo desarrollo P1. Si la disponibilidad o predisposición del equipo es baja, cinco días deja de ser un plazo fiable.

CONTRATOS Y PRUEBAS DEL PROTOTIPO

Referencia para T01/T04/T08/T10/T11/T13/T14/T16. Los tipos de esta pestaña son el contrato compartido; las matemáticas están en CU02 y el alcance en Objetivo y Modelo. Números decimales usan punto en código/JSON y coma en explicaciones.

1. Diccionario de entrada CSV

Las primeras nueve columnas son obligatorias y no admiten vacío. Las otras dos pueden omitirse; vacío se convierte a null. En el objeto interno ambos auxiliares siempre existen como number|null. Los IDs conservan ceros iniciales. No crear empleados automáticamente.

CSV	TypeScript	Tipo	Significado / regla
employee_id	employeeId	string	Código existente y activo en seed. No convertir a número.
period_start	periodStart	FechaISO	Lunes de semana cerrada.
period_end	periodEnd	FechaISO	Domingo, inicio+6 días.
horas_extra	horasExtra	number	0..168; decimal. HE.
dias_consecutivos_sin_descanso	diasConsecutivosSinDescanso	integer	≥0; racha al cierre, puede superar 7. DS.
tareas_activas	tareasActivas	integer	Total semanal, incluidas completadas. A≥0.
tareas_pendientes	tareasPendientes	integer	Sin completar al cierre. 0≤P≤A.
tareas_vencidas	tareasVencidas	integer	Pendientes con fecha límite ya vencida. 0≤V≤P.
vacaciones_pendientes_dias	vacacionesPendientesDias	integer	Saldo administrativo ≥0. VP.
horas_trabajadas	horasTrabajadas	number|null	Opcional; 0..168 y ≥HE.
dias_desde_ultimas_vacaciones	diasDesdeUltimasVacaciones	integer|null	Opcional; ≥0. Contexto, sin peso.

Huella de datos y mapeo

Normalizar antes de comparar: trim de celdas; ID sensible a mayúsculas; fechas ISO sin hora; números finitos en base decimal; -0 se normaliza a 0; auxiliares ausentes o vacíos son null. Serializar en el orden exacto de la tabla, con claves camelCase y valores normalizados; SHA-256 del JSON UTF-8 produce fingerprint. Distinto orden de filas/columnas, BOM, saltos de línea o 8 frente a 8.0 no cambian el contenido de un registro. sourceHash es SHA-256 de los bytes del archivo y sirve como metadato; no sustituye esta comparación.

2. Tipos compartidos y servicios

Ubicación acordada: src/shared/contracts.ts (tipos y errores), src/server/services (servicios), src/server/repositories (Prisma), src/app/api (rutas), src/app (vistas), templates (CSV), tests (datos y verificaciones). Cada módulo importa esos tipos; no mantener copias independientes.

```
type FechaISO = string; // YYYY-MM-DD validado en servidor
type Factor = 'HE' | 'DS' | 'CT' | 'VP';
type Nivel = 'BAJO' | 'MEDIO' | 'ALTO' | 'INCOMPLETO';
type EstadoJob =
  'PENDIENTE' | 'PROCESANDO' | 'COMPLETADO' | 'FALLIDO';
type ParPeriodo = {
  employeeId: string;
  periodStart: FechaISO;
  periodEnd: FechaISO;
};
type DatosLaborales = ParPeriodo & {
  horasExtra: number;
  diasConsecutivosSinDescanso: number;
  tareasActivas: number;
  tareasPendientes: number;
  tareasVencidas: number;
  vacacionesPendientesDias: number;
  horasTrabajadas: number | null;
  diasDesdeUltimasVacaciones: number | null;
};
type RegistroLaboral = DatosLaborales & {
  recordId: string;
  sourceImportId: string;
  dataVersion: number;
  fingerprint: string;
};
type ConfiguracionIndice = {
  version: string;
  effectiveFrom: FechaISO;
  windowDays: 7;
  persistencePeriods: 2;
  weights: Record<Factor, number>;
```

```
  thresholds: Record<Factor, number>;
  mediumLimit: number;
  highLimit: number;
  minExceededFactors: 2 | 3 | 4;
  createdBy: string;
  createdAt: string; // ISO UTC
};
type Persistencia = {
  anterior: null | {
    periodo: ParPeriodo;
    indice: number;
    excedidas: Factor[];
    dataFingerprint: string;
  };
  comunesExcedidas: Factor[];
  cumple: boolean;
  motivo: 'HISTORIAL_INSUFICIENTE'
    | 'NO_SOSTENIDO' | 'SOSTENIDO';
};
type Calculo = {
  indice: number;
  nivel: Nivel;
  indicadores: Record<Factor, number>;
  scores: Record<Factor, number>;
  contribuciones: Record<Factor, number>;
  ranking: Factor[];
  excedidas: Factor[];
  persistencia: Persistencia;
};
type Evaluacion = Calculo & ParPeriodo & {
  evaluationId: string;
  rulesVersion: string;
  engineVersion: 'cu02-v1';
  processingFingerprint: string;
  snapshot: {
    actual: RegistroLaboral;
    anterior: RegistroLaboral | null;
    reglas: ConfiguracionIndice;
  };
  supersedesEvaluationId: string | null;
  vigente: boolean;
  createdAt: string;
};
type Alerta = {
  alertId: string;
  evaluationId: string;
  employeeId: string;
  destinatarioId: string | null;
  motivoDestinatario: 'OK' | 'SIN_DESTINATARIO';
  causas: Factor[];
  sugerencias: string[];
  vigencia: 'ACTIVA' | 'SUPERADA';
  canal: 'INTERNO' | 'SIMULADO';
```

```
  estadoEnvio: 'NO_APLICA' | 'PENDIENTE' | 'ENVIADO'
    | 'FALLIDO' | 'SIN_DESTINATARIO';
  createdAt: string;
};
```

Reglas adicionales: vigente es un puntero por empleado/semana, no por rulesVersion; conservar versiones anteriores. anterior en Persistencia contiene el cálculo numérico con las reglas de la semana actual. cumple se refiere a las condiciones de ALTO (dos índices y factores comunes); motivo=NO_SOSTENIDO si hay anterior pero no se cumplen todas. Lista vacía de excedidas no es un dato faltante.

```
type ErrorCampo = {
  fila: number | null; // encabezado=1, primera fila=2
  campo: string | null; // nombre CSV si corresponde
  codigo: string;
  detalle: string;
};
type VistaPrevia = {
  importId: string;
  status: 'VALIDADA' | 'RECHAZADA';
  expiresAt: string;
  totalFilas: number;
  nuevas: number;
  omitidas: number;
  muestra: DatosLaborales[]; // primeras 10 válidas
  errores: ErrorCampo[];
  warnings: ErrorCampo[];
};
type JobDTO = {
  jobId: string;
  estado: EstadoJob;
  totalPares: number;
  completados: number;
  errores: { par: ParPeriodo; codigo: string;
    detalle: string }[];
  updatedAt: string;
};
type ResultadoImportacion = {
  importId: string;
  status: 'CONFIRMADA';
  insertados: number;
  omitidos: number;
  afectados: ParPeriodo[]; // nuevos y sucesores existentes
  procesamiento: JobDTO | null; // null si no hay afectados
};
type FactorInforme = {
  codigo: Factor;
  valor: number;
  umbral: number;
  score: number;
  contribucion: number;
  excedido: boolean;
```

```
};
type InformeDTO = {
  evaluacion: Evaluacion;
  empleado: { employeeId: string; nombre: string;
    teamId: string; tipo: 'TECNICO' | 'TEAM_LEAD' };
  factores: FactorInforme[]; // orden de ranking
  explicacion: string;
  recomendaciones: string[];
  alerta: Alerta | null;
};
type FilaTablero = ParPeriodo & {
  teamId: string;
  nombre: string;
  processingState: EstadoJob;
  actualizado: boolean;
  evaluationId: string | null;
  indice: number | null;
  nivel: Nivel | null;
  alertId: string | null;
};
type ResumenDTO = {
  totalEvaluaciones: number;
  conteo: Record<Nivel, number>;
  porcentajes: Record<Nivel, number>;
  noProcesados: number;
  promedioIndice: number | null;
  muestraPromedio: number;
};
type Pagina<T> = {
  items: T[];
  total: number;
  page: number;
  pageSize: number;
};
type ApiError = {
  error: { codigo: string; mensaje: string;
    detalles: ErrorCampo[] };
};
```

FilaTablero usa indice/nivel null solo cuando no hay evaluación; Evaluacion siempre tiene índice numérico. processingState describe ese par, no todos los pares del job: uno ya completado sigue COMPLETADO aunque falle otro. actualizado compara datos, antecedente, reglas y engineVersion esperados con el snapshot vigente. Si es false, señalar pendiente/fallo y no presentar el nivel anterior como actualizado; el detalle histórico sí puede conservarlo identificado.

Firmas obligatorias

```
// CU02 puro: sin acceso a base ni HTTP.
calcularEvaluacion(
  actual: RegistroLaboral,
  anterior: RegistroLaboral | null,
```

```
  reglas: ConfiguracionIndice
): Calculo

// Servicios asíncronos de servidor.
validarImportacion(
  archivo: File, actorId: string
): Promise<VistaPrevia>
confirmarImportacion(
  importId: string, actorId: string
): Promise<ResultadoImportacion>
procesarImportacion(jobId: string): Promise<JobDTO>
construirInforme(
  evaluationId: string
): Promise<InformeDTO>
crearAlertaSiCorresponde(
  evaluationId: string
): Promise<Alerta | null>
```

T07 implementa el motor puro; T08 lo usa, persiste y llama a CU03. T10 construye el informe; T11 crea/supera alertas. T04 orquesta confirmación/procesamiento; T13 expone las lecturas. construirInforme no crea alertas por una consulta GET: las alertas se generan en el procesamiento y el informe las lee. Error de contrato/regla lanza un error de dominio tipado que T04/T08 convierten a código, sin inventar Calculo.

3. API HTTP P0

Base /api; respuestas JSON excepto carga multipart. No crear rutas públicas para ejecutar CU02 empleado por empleado: lo ejecuta el servicio tras confirmar. Un 2xx al confirmar certifica la importación; consultar procesamiento.estado para saber si terminaron los cálculos.

Método / ruta	**Entrada**	**Éxito**	**Implementa / consume**
POST /api/importaciones/validar	multipart/form-data: file (CSV)	200 VistaPrevia; 422 vistaPrevia+error si filas inválidas	T04 / T06
POST /api/importaciones/{id}/confirmar	JSON {}	200 ResultadoImportacion	T04 / T06
DELETE /api/importaciones/{id}	Sin cuerpo	200 {importId,status:'CANCELADA'}	T04 / T06
GET	Sin cuerpo	200 JobDTO	T04 / T06,T14

/api/procesamientos/{jobId}			
POST /api/procesamientos/{jobId}/reintentar	JSON {}	200 JobDTO tras intento	T04+T08 / T06
GET /api/evaluaciones	Filtros/paginación	200 Pagina<FilaTablero>	T13 / T14
GET /api/evaluaciones/{evaluationId}	ID de evaluación	200 InformeDTO	T13+T10 / T14
GET /api/resumen	Filtros comunes	200 ResumenDTO	T13 / T14
GET /api/alertas	Filtros/paginación	200 Pagina<Alerta>	T13+T11 / T14

Consultas y errores uniformes

GET /api/evaluaciones: filtros opcionales employeeId, teamId, periodStart y nivel; page=1 y pageSize=50 por defecto, máximo 100. Orden periodStart descendente y employeeId ascendente. Devolver una FilaTablero por registro vigente, incluso pendiente/fallido; filtro nivel excluye filas sin nivel actualizado. GET /api/alertas: filtros opcionales employeeId, teamId, periodStart; page/pageSize iguales, orden createdAt descendente y alertId ascendente. Por defecto solo ACTIVA asociada a evaluación vigente y actualizada; el detalle de evaluación histórica muestra su alerta SUPERADA. GET /api/resumen usa employeeId/teamId/periodStart, sin paginación ni filtro nivel, para conservar el denominador. Filtros mal formados: 400.

HTTP 400: petición/formato inválido; 404: ID inexistente; 409: conflicto, job reclamado o estado incompatible; 410: vista previa vencida; 413: tamaño excedido; 415: formato no CSV; 422: filas/configuración inválidas; 500: error inesperado sin resultado controlado. ApiError es el formato común. Validar filas inválidas devuelve 422 {vistaPrevia:VistaPrevia,error:ApiError['error']}. Antes de parsear no es obligatorio crear importId. Confirmación/reintento procesados con fallo controlado devuelven 200 con job FALLIDO y sus errores; la UI no muestra éxito de cálculo por el 200. P1 agrega 401 sin sesión y 403 sin permiso.

Códigos de fila: OBLIGATORIO, NUMERO_INVALIDO, RANGO_INVALIDO, FECHA_INVALIDA, SEMANA_INVALIDA, PERIODO_ABIERTO, EMPLEADO_NO_EXISTE, EMPLEADO_FUERA_COBERTURA, DUPLICADO_LOTE, CONFLICTO_HISTORICO. Warnings: HORAS_EXTREMAS. Un warning nunca habilita una fila con error. En confirmación un conflicto sobrevenido devuelve 409 sin insertar y exige validar otra vez. Repetir un importId

CONFIRMADA devuelve su resultado, incluso si pasaron 30 minutos; no vuelve a procesar.

4. Estados, reintentos y cambios

Importación y procesamiento

Validar: VALIDADA o RECHAZADA. VALIDADA puede pasar a CANCELADA o CONFIRMADA. Los datos temporales expiran a los 30 minutos; los metadatos permanecen y un importId no confirmado vencido devuelve 410. CONFIRMADA no vuelve atrás. Cancelar una carga ya cancelada devuelve el mismo estado; cancelar una confirmada devuelve 409.

Confirmar inserta registros y job PENDIENTE en la misma transacción. afectados es la unión SIN DUPLICADOS de los pares nuevos y de sus sucesores inmediatos existentes, incluidos los nuevos del mismo lote. El job conserva esa lista y el estado por par. Después el servidor reclama job → PROCESANDO y ejecuta por empleado/fecha ascendente: cargar versiones actuales → calcular/persistir T08 → crear/superar alerta T11 → producir informe T10. Solo después el par cuenta completado. El informe es una proyección de Evaluacion; no requiere otra tabla.

Al primer error se detiene y el job queda FALLIDO con par/código; los resultados de pares ya terminados se conservan. Reintentar reclama el job de forma exclusiva y vuelve a ejecutar pares idempotentemente; finaliza COMPLETADO al completar todos. La ejecución usa lease de 120 segundos renovado antes de cada par y token de propietario: no se admite dos ejecutores a la vez. Job PROCESANDO con lease vigente devuelve 409 al reintentar; vencido puede reclamarse. Un ejecutor que perdió el token no puede publicar resultados ni cambiar el estado.

Al reiniciar, los jobs PENDIENTE o con lease vencido se muestran recuperables y ofrecen Reintentar. No depender de una promesa suelta que siga ejecutándose tras responder HTTP. P0 procesa dentro de la petición hasta terminar o fallar; límites de archivo acotan el trabajo. Un despliegue con límites de ejecución requerirá worker/cola posterior.

Correcciones y versiones

P0 no permite reemplazar registros conflictivos desde la UI ni borrar historial para reimportar. El operador corrige el archivo ANTES de confirmar. Sustitución de una carga confirmada queda para el sistema final: RRHH autorizado deberá registrar motivo y versión nueva, preservar anterior y recalcular semana corregida y sucesora. Esta funcionalidad no se da por implementada en T04.

Nuevas reglas P0 por script: Eros define, Gonzalo persiste; effectiveFrom es lunes y único. El script valida, crea versión y job con todas las semanas donde pasa a ser aplicable. Si una regla más reciente tiene effectiveFrom posterior, limita el intervalo de aplicación de la nueva. Las consultas señalan desactualizados hasta completar

reevaluación; no borran resultados anteriores. Nueva alerta ALTO supera a la anterior de esa semana. No implementar editor web ni correcciones manuales ocultas.

Concurrencia: antes de publicar, comprobar recordId/dataVersion actuales, configuración aplicable y antecedente contra lo leído para calcular; si cambiaron, no actualizar vigente y registrar REQUIERE_REPROCESO. El reintento toma las versiones nuevas. Transacción actualiza puntero vigente y supera la alerta antigua; el par no se marca completo hasta que su nueva alerta, si corresponde, existe.

5. Ejemplo integral reproducible

Seed: EMP-001 es técnico del equipo EQ-01; su responsable es TL-01, quien tiene superior GER-01. Todos ficticios. Configuración v1; reloj de prueba posterior al 2026-10-18 en la zona del prototipo. Las dos semanas siguientes están cerradas.

```
employee_id,period_start,period_end,horas_extra,dias_consecutivos_sin_descanso,tareas_activas,tareas_pendientes,tareas_vencidas,vacaciones_pendientes_dias,horas_trabajadas,dias_desde_ultimas_vacaciones
EMP-001,2026-10-05,2026-10-11,16,9,10,7,5,14,56,120
EMP-001,2026-10-12,2026-10-18,16,16,10,7,5,14,56,127
```

Paso 1 · Validar: nuevas=2, omitidas=0, errores=[], totalFilas=2; sin registros laborales aún. Paso 2 · Confirmar: insertados=2, omitidos=0 y dos pares afectados. Paso 3 · Procesar: primera semana 83,71; segunda 91,21. La racha DS pasa de 9 a 16 días porque transcurren siete jornadas más sin descanso. DS no se suma a otros indicadores ni se limita a siete.

Semana	Índice y nivel	Persistencia	Alerta
05–11/10/2026	83,71 · INCOMPLETO	Sin semana previa; guarda índice y cuatro excedidas	No
12–18/10/2026	91,21 · ALTO	Anterior=83,71; comunes HE,DS,CT,VP; cumple=true	Una, para TL-01

Informe de la segunda semana: HE 16/8, DS 16/6, CT 0,58/0,35, VP 14/10; scores 100/100/82,857142…/70; contribuciones 30/30/20,714285…/10,5; ranking HE→DS→CT→VP (HE precede a DS por desempate fijo). Explicación: «91,21 puntos; dos semanas consecutivas ≥70 y cuatro factores comunes excedidos». Sugerencias: HE revisar horas extra; DS descansos; CT pendientes/vencimientos; VP planificación del saldo. Revisión humana, sin decisiones automáticas.

Tablero: primera semana INCOMPLETO con 83,71; segunda ALTO con 91,21, alerta ACTIVA dirigida a TL-01, canal INTERNO y NO_APLICA. No enviar correo. Resumen de la segunda semana: total=1, ALTO=1 (100%), demás=0, noProcesados=0, promedio=91,21, muestraPromedio=1.

Repetir el mismo archivo: nuevas=0, omitidas=2; confirmar insertados=0 y procesamiento=null. Permanecen dos registros, dos evaluaciones y una alerta. Repetir el mismo importId confirmado devuelve el resultado original, con insertados=2; no representa otra inserción.

6. Escenarios de aceptación compartidos

Caso	**Entrada / acción**	**Resultado esperado**
BAJO	Dos semanas: HE=DS=VP=0; A=P=V=0	Primera INCOMPLETO índice 0; segunda BAJO 0; sin alerta.
MEDIO	Dos semanas: HE=8,DS=6,A=10,P=5,V=2,VP=10	CT=0,32; índice 48,93; segunda MEDIO; sin alerta.
ALTO	Ejemplo de §5	83,71 y 91,21; segunda ALTO y una alerta.
Pico aislado	Anterior: HE=0,DS=2,A=P=V=VP=0 (índice 5). Actual: HE=16,DS=9,A=10,P=7,V=5,VP=14.	Segunda MEDIO 83,71, NO_SOSTENIDO; sin alerta.
Hueco	Datos del 05/10 y del 19/10, falta 12/10	19/10 INCOMPLETO con índice numérico.
Fuera de orden	Importar 12/10 alto y después 05/10 alto	12/10 primero INCOMPLETO; luego ALTO vigente nuevo; anterior conservada; una alerta ACTIVA.
Archivo inválido	Una fila sin HE o con V>P	422 y cero registros/evaluaciones nuevos en todo el lote.
Duplicado / conflicto	Repetido interno; histórico distinto	Interno 422; conflicto detectado al validar 422, sobrevenido al confirmar 409; cero sustituciones.
Idempotencia	Repetir archivo, confirmar ID y reintentar job	No duplicar registros, evaluaciones ni alertas; conteos según §5.
Cero y límites	A=P=V=0; A=0,P=1; Xi=Ui	CT=0; segundo inválido; igualdad con umbral no excedida.
Redondeo /	Índice contractual 39,99/40/69,99/70 y	Con historia: 39,99 BAJO; 40/69,99 MEDIO; 70 ALTO solo si persiste. Empate

ranking	contribuciones empatadas	HE/DS/CT/VP. Entradas exactas de borde en §6.1.
Job fallido	Forzar fallo T11 después de guardar datos/evaluación; reintentar	Carga CONFIRMADA, job FALLIDO; al reintentar COMPLETADO, sin duplicados.
Destinatario / vigencia	ALTO sin responsable; nueva evaluación no ALTO	Incidencia visible; alerta antigua SUPERADA y ninguna nueva ACTIVA.
Reglas nuevas	Nueva versión válida por script para semanas existentes	Job recalcula intervalo, conserva historial y actualiza vigentes; regla inválida no se guarda.

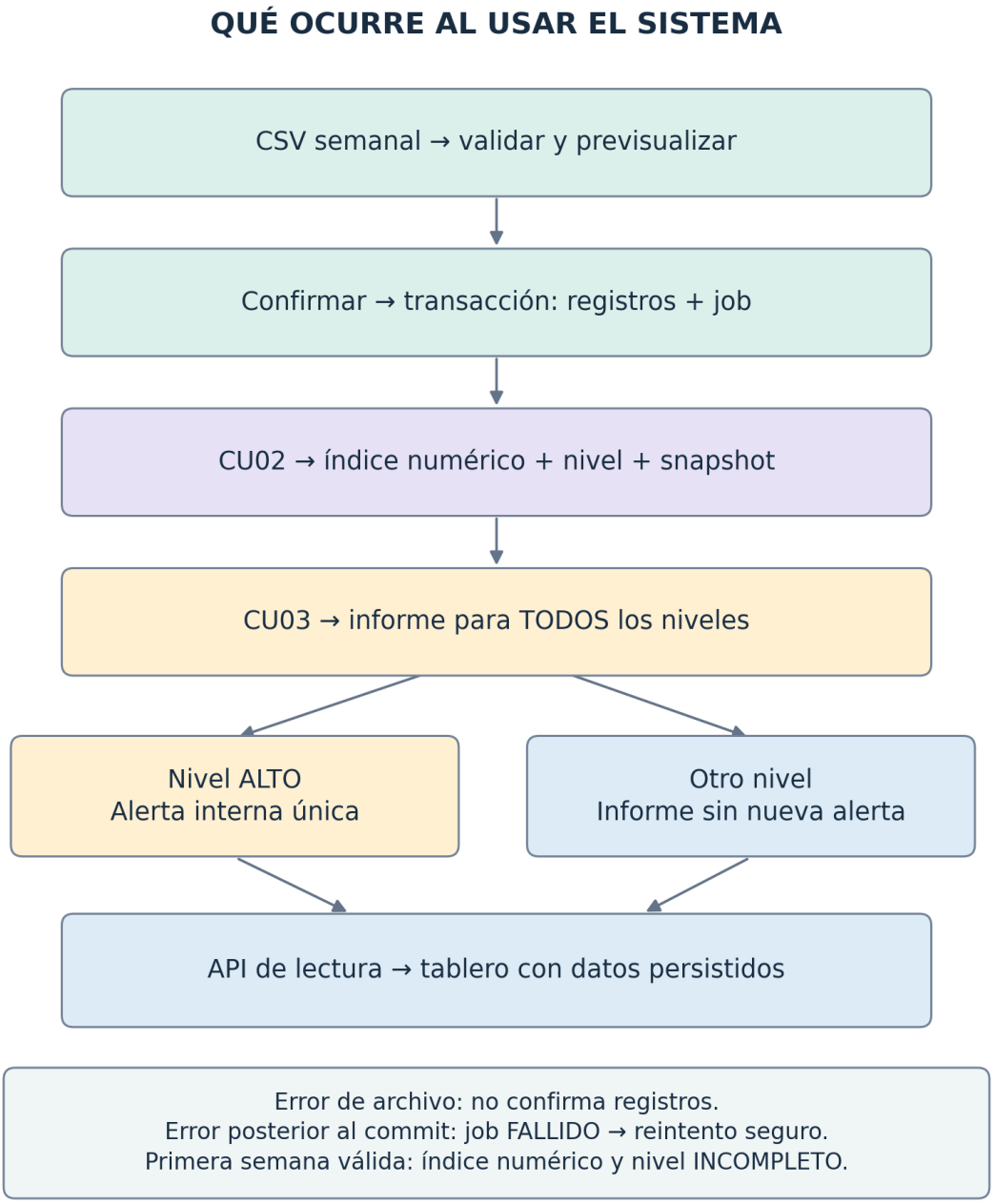
6.1 Entradas exactas para probar los límites

Cada fila usa reglas v1 y semana anterior con los mismos valores, salvo DS: para las filas de 69,99 y 70, DS anterior=12 y actual=19. Son dos semanas consecutivas; los auxiliares pueden ser null. A/P/V es total/pendientes/vencidas. Comparar el índice redondeado contractual.

HE	**DS actual**	**A / P / V**	**VP**	**Índice / nivel**
9.328	6	0 / 0 / 0	10	39,99 · BAJO
9.3333	6	0 / 0 / 0	10	40,00 · MEDIO
3.9946667	19	1 / 1 / 1	10	69,99 · MEDIO
4	19	1 / 1 / 1	10	70,00 · ALTO

7. Flujo de ejecución del sistema

Este flujo representa lo que ocurre al usar la app. El mapa de Tareas Resumen representa dependencias para desarrollar y terminar tareas; no confundir ambos. Los diagramas omiten responsables.



8. Glosario y criterio de entrega

DTO: objeto que un módulo entrega a otro. Contrato: forma y significado obligatorio de entradas/salidas. Seed: datos iniciales ficticios y reglas. Fingerprint: huella estable de contenido normalizado. Idempotencia: repetir una operación sin duplicar su efecto. Snapshot: copia de los datos/reglas usados. Transacción: guardado íntegro o rollback. Vigente: resultado actual por empleado/semana. Job: trabajo persistido de procesamiento. Lease: permiso temporal exclusivo para ejecutarlo. Mock: respuesta simulada con el mismo contrato. E2E: prueba desde carga del archivo hasta resultado visible.

Hecho: contrato compartido usado, criterios de la tarea cumplidos, pruebas relevantes aprobadas, npm run build correcto y PR revisado. README permite instalar, migrar, seedear, ejecutar y repetir el ejemplo sin instrucciones verbales; incluir .env.example, versiones en lockfile, comandos de pruebas y recuperación de job. T16 valida instalación limpia. Cada cambio contractual se actualiza en consumidores y pruebas antes de integrar.